HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN

BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN
NĂM 2026

NGHIÊN CỨU MÔ HÌNH ĐÁNH GIÁ RỦI RO TRỄ HẠN CÔNG VIỆC TRONG SPRINT VÀ XÂY DỰNG AGENT HỖ TRỢ CAN THIỆP SỚM

04-SV-2026-RIPT

Thuộc nhóm ngành khoa học: Công nghệ thông tin

Sinh viên thực hiện	Giới tính	Dân tộc	Mã SV	Lớp	Khoa	Năm thứ	Số năm đào tạo	Ngành học
Nguyễn Trường Giang	Nam	Kinh	B23DCCC055	D23CCPM02	Viện Khoa học Kỹ thuật Bưu điện	4	4	Cử nhân Công nghệ thông tin Định hướng ứng dụng

Người hướng dẫn: ThS. Trần Minh Hiếu

Hà Nội - 2026

TÓM TẮT

Trong một sprint, nhiều issue không kịp hoàn thành nhưng người quản lý thường chỉ nhận ra khi sprint đã gần kết thúc. Trong đề tài này, chúng tôi xây dựng một hệ thống trên bộ dữ liệu TAWOS [6] (dữ liệu Jira của các dự án mã nguồn mở) để trả lời lần lượt ba câu hỏi: issue nào có nguy cơ trễ hạn, vì sao, và nên can thiệp như thế nào. Khối phát hiện sử dụng mô hình CatBoost [5] dựa trên tiến độ công việc đã diễn ra trước mốc thời gian t trong sprint. Khối chẩn đoán chạy năm truy vấn SQL cố định để thu thập các sự kiện trước mốc t, gọi LLM một lần để đề xuất tối đa ba giả thuyết nguyên nhân kèm mã sự kiện làm bằng chứng, rồi dùng bước kiểm tra bằng code (verifier) loại bỏ mọi giả thuyết không khớp sự kiện. Hệ thống chỉ dừng ở mức đề xuất; mọi thay đổi trên ứng dụng quản lý công việc Plane chỉ diễn ra khi người dùng bấm Duyệt và đều có thể Hoàn tác. Trên các dự án chưa từng xuất hiện khi huấn luyện, ở mốc 50% thời lượng sprint, cảnh báo của mô hình đạt độ chính xác (precision) 76,3% so với tỉ lệ nền 67,0% (lift đạt 1,14). Trên 81 cảnh báo của dự án IS (Indy SDK), nguyên nhân đứng đầu do hệ thống đưa ra khớp với nhãn của người chấm độc lập ở 61,7% số cảnh báo ($\kappa = 0{,}50$); trên tập 60 issue kiểm định độc lập (held-out) sau khi điều chỉnh quy tắc sắp xếp nguyên nhân, tỉ lệ khớp đạt 68,3% ($\kappa = 0{,}57$). Ở cấp sprint, số lượng issue không kịp hoàn thành được ước lượng lệch trung bình 1,76 issue mỗi sprint, nhưng danh sách issue cụ thể được chọn để chuyển sang sprint sau chỉ đạt precision 52,8% so với mức 48,8% khi chọn ngẫu nhiên. Với agent hỏi đáp, cả 25 câu hỏi kiểm thử đều được trả lời đúng phạm vi hoặc từ chối hợp lệ, bao gồm 3 câu tấn công tiêm lệnh. Cuối báo cáo, chúng tôi phân tích các giới hạn hiện tại của đề tài: nhãn đối chiếu do một mô hình độc lập gán, thực nghiệm chẩn đoán mới chạy trên một dự án và chưa có thử nghiệm trực tiếp với người dùng thật.

Từ khóa: rủi ro sprint, cảnh báo sớm, giám sát quy trình dự đoán, chẩn đoán có bằng chứng, issue tracker, LLM, con người duyệt.

MỤC LỤC

TÓM TẮT i
DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ vi
CHƯƠNG 1. GIỚI THIỆU 1
1.1. Bối cảnh 1
1.2. Bài toán 1
1.3. Câu hỏi nghiên cứu 1
1.4. Đóng góp 2
1.5. Phạm vi 3
1.6. Cấu trúc báo cáo 3
CHƯƠNG 2. TỔNG QUAN VÀ ĐỊNH VỊ 4
2.1. Giám sát quy trình dự đoán và dữ liệu lịch sử issue 4
2.2. Agent chủ động và bài toán quyết định khi nào nên báo 5
2.3. Giải thích có căn cứ và sử dụng công cụ an toàn 5
2.4. Ước lượng công sức và lý do chọn cách tiếp cận cấp sprint 6
CHƯƠNG 3. BÀI TOÁN VÀ DỮ LIỆU 7
3.1. Định nghĩa bài toán 7
3.2. Dữ liệu TAWOS 8
3.3. Quy tắc cắt tại mốc t 8
3.4. Các giả định về dữ liệu 9
CHƯƠNG 4. PHƯƠNG PHÁP 10
4.1. Khối phát hiện rủi ro 10
4.2. Bộ thu thập sự kiện bằng năm truy vấn SQL 11
4.3. Chẩn đoán nguyên nhân bằng một lần gọi LLM 11
4.4. Bước hậu kiểm bằng code (Verifier) 12
4.5. Hai ghi chú bối cảnh theo luật 12
4.6. Thực thi hành động và cơ chế Duyệt / Bỏ qua / Hoàn tác 13
4.7. Lập kế hoạch cấp sprint (Module M14) 13
4.8. Duyệt và hoàn tác theo lô (Module M15) 14
4.9. Agent hỏi đáp chỉ đọc (Module M16) 14
4.10. Sổ cảnh báo theo dõi vòng đời (Module M17) 15
4.11. Bảy bất biến an toàn của hệ thống 15
CHƯƠNG 5. HIỆN THỰC HỆ THỐNG 17
5.1. Kiến trúc hệ thống 17
5.2. Giao diện người dùng trên Plane 18
5.3. Minh họa luồng xử lý qua một cảnh báo cụ thể 24

5.4. Kiểm thử tự động....26
CHƯƠNG 6. THIẾT KẾ THỰC NGHIỆM....27
CHƯƠNG 7. KẾT QUẢ THỰC NGHIỆM....29
7.1. Kết quả khối phát hiện rủi ro (RQ1)....29
7.2. Kết quả khối chẩn đoán nguyên nhân (RQ2)....31
7.3. Kết quả khối lập kế hoạch cấp sprint (RQ3)....34
7.4. Kết quả đánh giá agent hỏi đáp (RQ4)....35
7.5. Chi phí vận hành và hiệu năng hệ thống....36
CHƯƠNG 8. THẢO LUẬN....38
CHƯƠNG 9. MỐI ĐE DỌA TÍNH HỢP LỆ VÀ HẠN CHẾ....40
9.1. Tính hợp lệ nội tại (Internal validity)....40
9.2. Tính hợp lệ ngoại tại (External validity)....40
9.3. Tính hợp lệ của cấu trúc đo (Construct validity)....41
9.4. Tính hợp lệ của kết luận (Conclusion validity)....41
CHƯƠNG 10. KẾT LUẬN VÀ KIẾN NGHỊ....42
10.1. Kết luận....42
10.2. Kiến nghị và hướng phát triển....42
TÀI LIỆU THAM KHẢO....44
PHỤ LỤC....45
Phụ lục A. Prompt hệ thống dùng cho bước chẩn đoán....45
Phụ lục B. Lược đồ dữ liệu đầu ra và cấu trúc thẻ cảnh báo....45
Phụ lục C. Quy tắc chấm nhãn độc lập (Module M13)....46
Phụ lục D. Các kịch bản thực thi để tái lập kết quả thực nghiệm....46
Phụ lục E. Danh mục các bộ kiểm thử tự động....47

DANH MỤC BẢNG

Bảng 3.1. Các bước lọc từ dữ liệu gốc đến tập đánh giá khối phát hiện 8
Bảng 4.1. Bảy bất biến an toàn và nơi bảo đảm trong hệ thống 16
Bảng 5.1. Các khối chức năng và công nghệ sử dụng trong hệ thống 17
Bảng 6.1. Ma trận câu hỏi nghiên cứu, thiết kế thực nghiệm và quy tắc kết luận.....27
Bảng 7.1. So sánh Recall của mô hình tĩnh và mô hình động theo từng mốc thời gian29
Bảng 7.2. Chất lượng cảnh báo trên tập issue còn mở theo chính sách vận hành thực tế (E8, E9).....30
Bảng 7.3. Độ đồng thuận giữa chẩn đoán của hệ thống và nhãn của người chấm (Module M13) 32
Bảng 7.4. Thống kê chi tiết theo từng nhãn nguyên nhân (Số lần hệ thống xếp đầu / Số lần người chấm gán / Số lần trùng khớp) 33
Bảng 7.5. Kết quả backtest kế hoạch cấp sprint trên 60 sprint của dự án IS tại mốc 50% 34
Bảng 7.6. Kết quả kiểm thử 25 câu hỏi kịch bản trên agent hỏi đáp (Module M16)....36
Bảng B.1. Lược đồ JSON phản hồi của LLM (kiểm tra tự động bằng Pydantic) 45
Bảng C.1. Định nghĩa các nhãn nguyên nhân và điều kiện gán nhãn 46
Bảng D.1. Các kịch bản thực thi và kết quả đầu ra tương ứng.....47
Bảng E.1. Các tệp kiểm thử tự động và nội dung kiểm tra tương ứng.....47

DANH MỤC HÌNH

Hình 3.1. Quy tắc cắt tại mốc t (minh họa ở mốc 50% thời lượng sprint): chỉ các sự kiện nằm bên trái mốc t được đưa vào mô hình và bộ thu thập..... 8

Hình 5.1. Sáu khối chức năng của hệ thống; mũi tên đỏ biểu thị luồng ghi duy nhất từ dịch vụ risk sang API của Plane khi người dùng bấm Duyệt.....17

Hình 5.2. Thẻ cảnh báo trên trang chi tiết issue ISP-401 trong Plane (lượt chạy thứ hai); phía trên là bình luận tóm tắt do tiến trình tự động đăng 19

Hình 5.3. Màn hình Delivery risk trong mục Analytics (lượt chạy thứ hai), hiển thị thống kê dự án và thẻ kế hoạch sprint.....20

Hình 5.4. Danh sách cảnh báo đang chờ phê duyệt trên trang chủ Home (lượt chạy thứ hai).....21

Hình 5.5. Màn hình Your work trống do dữ liệu ẩn danh (trên) và chế độ xem đã lưu "Flagged by the risk check" liệt kê 81 issue có cảnh báo (dưới) 22

Hình 5.6. Thẻ kế hoạch cấp sprint của "LibINDY 18.04 Revocation" (chế độ phát lại sprint lịch sử).....23

Hình 5.7. Kết quả sau khi bỏ chọn IS-546 và phê duyệt theo lô 5 dòng còn lại; nút Undo cho phép hoàn tác cả lô vừa duyệt.....23

Hình 5.8. Khung hỏi đáp Ask trong tab Delivery risk: câu trả lời dẫn chiếu mã issue cụ thể, dòng "Looked up" ghi nhận 1 lần gọi list_at_risk, 5 lần gọi explain_alert và chi phí 0,00063 USD 24

Hình 5.9. Luồng xử lý cảnh báo ISP-401 (IS-1119) qua năm bước trong hệ thống (lượt chạy thứ hai).....24

Hình 7.1. Diễn biến Recall khi giới hạn ngân sách cảnh báo 20% số issue mỗi sprint qua các mốc thời gian.....30

Hình 7.2. Chỉ số Lift của mô hình động (kèm khoảng tin cậy 95%) đối chiếu với quy tắc đơn giản qua các mốc thời gian (thực nghiệm E9) 31

Hình 7.3. Phân bố nguyên nhân đứng đầu trên 81 cảnh báo của dự án IS ở lượt chạy thứ nhất (trước khi điều chỉnh quy tắc sắp xếp thứ tự).....32

Hình 7.4. Ma trận nhầm lẫn giữa nhãn của người chấm và nguyên nhân đứng đầu của hệ thống: lượt 1 trên 81 cảnh báo (trái) và tập 60 issue held-out sau khi sửa quy tắc thứ tự (phải) 33

Hình 7.5. Đối chiếu số lượng issue không xong thực tế, số không xong dự kiến và số lượng trong tập chuyển qua 60 sprint của dự án IS tại mốc 50% 35

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ	Nghĩa trong báo cáo
AP	Average Precision, diện tích dưới đường cong precision-recall
API	Giao diện lập trình ứng dụng
BPM	Quản trị quy trình nghiệp vụ (Business Process Management)
CI	Khoảng tin cậy 95% (ước lượng bằng bootstrap)
E1-E9	Mã các thực nghiệm phát hiện rủi ro ở giai đoạn trước
IS / ISP	Dự án Indy SDK của Hyperledger trong TAWOS (mã IS) và bản nhập vào Plane (mã ISP)
JSON	Định dạng dữ liệu có cấu trúc dùng trao đổi giữa các thành phần
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
M0-M17	Mã các module phần mềm xây dựng trong đề tài
MAE	Sai số tuyệt đối trung bình (Mean Absolute Error)
Plane	Nền tảng quản lý dự án mã nguồn mở dùng để tích hợp giao diện (trong Plane, một sprint được gọi là một cycle)
RQ	Câu hỏi nghiên cứu (Research Question)
SQL	Ngôn ngữ truy vấn cơ sở dữ liệu có cấu trúc
TAWOS	Bộ dữ liệu Jira của các dự án Agile mã nguồn mở [6]
Backtest	Chạy kiểm nghiệm lại một quy tắc trên các sprint đã kết thúc rồi đối chiếu với kết quả thực tế
Held-out	Tập dữ liệu giữ riêng để kiểm định, không dùng khi tinh chỉnh hệ thống
Lift	Tỉ số giữa precision và tỉ lệ nền; lift bằng 1 tương đương chọn ngẫu nhiên
Verifier	Bước kiểm tra bằng code chạy sau LLM, loại bỏ phần đầu ra không khớp sự kiện lịch sử

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh

Ở đầu mỗi sprint, nhóm phát triển nhận một danh sách công việc (issue) và cam kết hoàn thành trước ngày kết thúc. Qua thống kê trên 18.322 cặp issue-sprint thuộc 14 dự án trong bộ dữ liệu TAWOS [6] mà chúng tôi sử dụng để đánh giá, có 47,21% trường hợp kết thúc sprint khi issue vẫn chưa hoàn thành. Các công cụ quản lý công việc như Jira hay Plane (nền tảng quản lý dự án mã nguồn mở tương tự Jira mà chúng tôi dùng để tích hợp giao diện) đều lưu lại lịch sử thao tác đủ chi tiết để nhận diện sớm nguy cơ trễ hạn: trạng thái đứng yên nhiều ngày, người phụ trách đang bận xử lý các issue khác, issue bị chặn bởi một issue chưa xong, hoặc phần thảo luận còn bỏ ngỏ. Tuy nhiên, khi quản lý một sprint gồm vài chục issue, trưởng nhóm khó có đủ thời gian đọc lại nhật ký hoạt động của từng issue mỗi ngày.

Ở giai đoạn đầu của đề tài (các thực nghiệm từ E1 đến E9), chúng tôi đã xây dựng và kiểm tra khối phát hiện: sử dụng mô hình CatBoost [5] để chấm điểm nguy cơ trễ hạn cho từng issue tại một mốc thời gian trong sprint. Dù vậy, một điểm số xác suất đơn lẻ chưa cho người quản lý biết vì sao issue đó gặp rủi ro và nên xử lý thế nào. Trong giai đoạn tiếp theo được trình bày ở báo cáo này (các module phần mềm từ M0 đến M17), chúng tôi lấy khối phát hiện làm nền tảng và phát triển phần còn lại của quy trình hỗ trợ quyết định: giải thích nguyên nhân bằng sự kiện lịch sử kiểm chứng được, đề xuất hành động ở cả cấp issue lẫn cấp sprint, và giữ quyền phê duyệt mọi thao tác ghi dữ liệu cho người dùng.

1.2. Bài toán

Tại một mốc thời gian t trong sprint (tính theo phần trăm thời lượng sprint đã trôi qua), với mỗi issue nằm trong danh sách cảnh báo, hệ thống cần chỉ ra nguyên nhân khả dĩ nhất trong một tập nguyên nhân đóng, trỏ trực tiếp tới các sự kiện có thật diễn ra trước hoặc tại mốc t; nêu rõ điều gì chưa biết và cần hỏi ai; đồng thời đề xuất một hành động trong tập đóng (nâng ưu tiên, nhắc người phụ trách, hoặc chuyển sang sprint kế tiếp) mà người dùng có thể Duyệt, Bỏ qua hoặc Hoàn tác. Ở cấp sprint, hệ thống ước lượng số không xong dự kiến trong số các issue còn mở và đề xuất tập chuyển sang sprint sau. Định nghĩa hình thức của bài toán được trình bày ở mục 3.1.

1.3. Câu hỏi nghiên cứu

Chúng tôi đặt ra bốn câu hỏi nghiên cứu tương ứng với bốn bước xử lý của hệ thống:

RQ1. Điểm phát hiện sớm có đủ độ tin cậy để làm căn cứ sắp xếp thứ tự cảnh báo hay không? Câu hỏi này kế thừa kết quả E1 đến E9 và đo lại theo đúng cơ chế hệ thống đang vận hành: chỉ xét issue còn mở tại mốc đánh giá, giới hạn số cảnh báo mỗi sprint, rồi đối chiếu với tỉ lệ nền và với một quy tắc sắp xếp đơn giản.

RQ2. Chẩn đoán của hệ thống có bám sát bằng chứng lịch sử hay không, và mức độ khớp với bộ nhãn của người chấm đạt bao nhiêu? Độ bám bằng chứng được đo bằng số giả thuyết bị verifier loại bỏ; độ khớp được đo trên 81 cảnh báo của dự án IS và 60 issue trong tập held-out (module M13).

RQ3. Ở cấp sprint, công thức thống kê có ước lượng sát số không xong dự kiến và chọn đúng những issue đó vào tập chuyển hay không? Chúng tôi đánh giá bằng phương pháp backtest trên lịch sử các sprint đã kết thúc (module M14).

RQ4. Agent hỏi đáp có trả lời đúng phạm vi quản lý rủi ro, từ chối các câu ngoài phạm vi, và bảo đảm không làm lộ dữ liệu của dự án mà người hỏi không có quyền xem hay không? Chúng tôi kiểm tra bằng bộ 25 câu hỏi cố định (module M16).

Câu hỏi liệu hệ thống có giúp giảm trễ hạn trực tiếp trong môi trường làm việc thực tế hay không đòi hỏi thử nghiệm có người tham gia và thiết kế can thiệp đối chứng; phần này nằm ngoài phạm vi đề tài và được thảo luận tại chương 9.

1.4. Đóng góp

Đề tài có bốn đóng góp thực nghiệm:

Thứ nhất, quy trình chẩn đoán đặt LLM giữa hai lớp kiểm soát bằng code: lớp trước (bộ thu thập) chỉ đưa vào các sự kiện đã cắt tại mốc t, lớp sau (verifier) loại bỏ mọi giả thuyết không trỏ tới sự kiện có thật.

Thứ hai, phép đo độ khớp giữa chẩn đoán của hệ thống với nhãn của người chấm trên 81 cảnh báo và 60 issue held-out, đi kèm phân tích cụ thể về sai lệch thứ tự ưu tiên và giới hạn của bộ nhãn.

Thứ ba, cơ chế lập kế hoạch cấp sprint tách bạch phần tính toán định lượng bằng code khỏi phần giải thích ngôn ngữ, cùng kết quả backtest trên 60 sprint chỉ ra rằng ước lượng số không xong dự kiến ở cấp sprint đáng tin cậy hơn việc chọn đích danh từng issue để chuyển lịch.

Thứ tư, bản tích hợp vào nền tảng Plane gồm thẻ cảnh báo, kế hoạch sprint, duyệt theo lô, sổ cảnh báo và agent hỏi đáp chỉ đọc, tuân thủ bảy bất biến an toàn (mục 4.11) có kiểm thử tự động.

Các kết quả trong báo cáo là bằng chứng thực nghiệm cho một bài toán kỹ thuật phần mềm trên dữ liệu lịch sử; chúng tôi không đề xuất thuật toán học máy mới.

1.5. Phạm vi

Chúng tôi sử dụng bộ dữ liệu TAWOS phiên bản 1.1 [6] nạp vào PostgreSQL. Khối phát hiện được đánh giá trên 14 dự án; phần chẩn đoán, kế hoạch sprint và giao diện người dùng được triển khai và đo kiểm trên dự án IS (Indy SDK của Hyperledger, một dự án bộ công cụ nhận dạng phi tập trung trong TAWOS), nhập vào Plane dưới mã dự án ISP. Hệ thống không ước lượng story point hay ngày hoàn thành của từng issue riêng lẻ (lý do thực nghiệm trình bày ở mục 2.4), không huấn luyện lại mô hình từ phản hồi Duyệt/Bỏ qua, không kết nối trực tiếp vào dữ liệu đang vận hành của một đội phát triển bên ngoài, và không dùng kiến trúc đa agent.

1.6. Cấu trúc báo cáo

Chương 2 tổng quan các hướng nghiên cứu liên quan và định vị đề tài. Chương 3 định nghĩa bài toán và mô tả dữ liệu thực nghiệm. Chương 4 trình bày phương pháp của từng thành phần trong hệ thống. Chương 5 mô tả kiến trúc cài đặt và minh họa luồng xử lý qua một cảnh báo cụ thể trên Plane. Chương 6 trình bày thiết kế thực nghiệm, chương 7 phân tích kết quả đo đạc, và chương 8 thảo luận các phát hiện chính. Chương 9 phân tích các mối đe dọa tính hợp lệ cùng hạn chế của nghiên cứu. Chương 10 tóm lược kết luận và nêu kiến nghị.

CHƯƠNG 2. TỔNG QUAN VÀ ĐỊNH VỊ

Chúng tôi khảo sát các công trình liên quan trực tiếp đến bốn quyết định thiết kế của hệ thống: giám sát quy trình dự đoán trên dữ liệu lịch sử issue, cơ chế kích hoạt agent chủ động, an toàn khi kết hợp LLM với công cụ và bài toán ước lượng công sức trong Agile.

2.1. Giám sát quy trình dự đoán và dữ liệu lịch sử issue

Bài toán dự báo một issue có kịp hoàn thành trong sprint hay không thuộc lĩnh vực giám sát quy trình dự đoán (Predictive Process Monitoring), một hướng nghiên cứu kết hợp khai phá quy trình (Process Mining) và quản trị quy trình nghiệp vụ (Business Process Management - BPM). Cách tiếp cận này sử dụng dữ liệu lịch sử cùng dữ liệu thời gian thực trích xuất từ nhật ký sự kiện (event logs) kết hợp với học máy để dự đoán diễn biến và kết quả tương lai của một quy trình đang hoạt động. Trong khảo sát về giám sát quy trình dự đoán định hướng kết quả (outcome-oriented predictive process monitoring) [8], mỗi lượt thực thi quy trình đang chạy được biểu diễn như một chuỗi sự kiện có tên hoạt động và thời điểm xảy ra; mô hình dựa trên tiến độ công việc đã diễn ra trước thời điểm đánh giá để dự đoán kết quả cuối cùng, và mười một phương pháp tiêu biểu đã được so sánh trên cùng bộ chuẩn chung. Cách đặt bài toán của chúng tôi khớp với khung hình thức đó: mỗi cặp issue-sprint tương ứng với một lượt thực thi quy trình, các thay đổi diễn ra trước mốc t là phần lịch sử đầu vào, và nhãn đầu ra là trạng thái hoàn thành hay không hoàn thành khi sprint kết thúc.

Bước tiếp theo sau dự đoán kết quả là giám sát quy trình theo hướng đề xuất (prescriptive process monitoring), tức dùng kết quả dự đoán để gợi ý hành động can thiệp ngay khi quy trình đang chạy. Nghiên cứu tổng quan về nhánh này [3] phân loại các phương pháp theo mục tiêu, hình thức can thiệp, dữ liệu đầu vào và chính sách kích hoạt can thiệp, đồng thời chỉ ra rằng phần lớn phương pháp mới dừng ở đánh giá ngoại tuyến chứ chưa được kiểm chứng trong môi trường làm việc thực tế. Đề tài của chúng tôi thuộc nhánh nghiên cứu này nhưng giới hạn ở mức đề xuất chờ người dùng duyệt, và yêu cầu lý do can thiệp phải dẫn chiếu tới các sự kiện cụ thể trong lịch sử issue thay vì chỉ dựa vào điểm số rủi ro.

Về dữ liệu thực nghiệm, bộ dữ liệu TAWOS [6] cung cấp lịch sử issue Jira của 44 dự án lấy từ 13 kho mã nguồn mở, bao gồm bảng sprint, nhật ký thay đổi (`change_log`), bình luận (`comment`) và liên kết phụ thuộc giữa các issue (`issue_link`); tên người dùng cùng địa chỉ email đều đã được ẩn danh. Phiên bản 1.1 mà chúng tôi nạp vào PostgreSQL gồm 39 dự án và 458.232 issue (số lượng dự án khác với bài báo gốc do khác phiên bản phát hành). Khi khai thác dữ liệu lịch sử từ các hệ thống quản lý issue, nghiên cứu thực

nghiệm về sai lệch thời gian [9] chỉ ra một lỗi phổ biến: trong dữ liệu Jira nói chung và TAWOS nói riêng, nhiều cột của bảng `issue` lưu giá trị cuối cùng sau khi issue đã đóng; nếu lấy trực tiếp các cột này làm đặc trưng tại một mốc thời gian trước đó thì mô hình đã dùng thông tin tương lai. Chúng tôi xử lý lỗi rò rỉ thời gian này bằng quy tắc cắt tại mốc t ở mục 3.3.

2.2. Agent chủ động và bài toán quyết định khi nào nên báo

Một hệ thống cảnh báo chủ động phải giải quyết hai câu hỏi tách biệt: khi nào cần lên tiếng và lên tiếng về nội dung gì. Nghiên cứu về cơ chế kích hoạt agent chủ động [4] chỉ ra rằng nếu dùng LLM cho cả quyết định "khi nào cần báo", hệ thống phải gọi LLM liên tục kể cả ở những thời điểm không cần can thiệp; cách thiết kế tiết kiệm và ổn định hơn là dùng một mô hình nhỏ riêng để quyết định thời điểm kích hoạt và chỉ gọi LLM khi đã xác định cần báo. Chúng tôi áp dụng cách tách vai trò này: quyết định cảnh báo issue nào do mô hình CatBoost [5] và quy tắc ngân sách cố định đảm nhận, bởi phần này tái lập được và kiểm tra được trên dữ liệu lịch sử (RQ1), còn LLM chỉ được gọi cho những issue đã nằm trong danh sách cảnh báo.

Số cảnh báo trong mỗi sprint được giới hạn bởi ngân sách $K = \lceil 0{,}2n \rceil$, với n là số issue đủ điều kiện xét duyệt trong sprint đó. Cách giới hạn theo ngân sách cố định sát với thực tế vận hành hơn so với dùng một ngưỡng xác suất cố định, vì cùng một ngưỡng xác suất có thể sinh ra quá nhiều cảnh báo ở dự án này nhưng lại không sinh cảnh báo nào ở dự án khác. Khi đánh giá theo ngân sách, độ chính xác của cảnh báo (precision) phải được đối chiếu với tỉ lệ nền của chính tập issue đủ điều kiện thông qua chỉ số lift; nếu chỉ nhìn vào precision tuyệt đối, một con số cao ở giữa hoặc cuối sprint có thể chỉ phản ánh việc phần lớn issue còn mở vào thời điểm đó vốn sẽ không kịp hoàn thành (mục 7.1).

2.3. Giải thích có căn cứ và sử dụng công cụ an toàn

Khi LLM đọc dữ liệu do người dùng viết (như bình luận hay mô tả issue), mô hình có nguy cơ làm theo các mệnh lệnh cài cắm bên trong dữ liệu đó. Nghiên cứu về bảo mật ứng dụng tích hợp LLM [2] gọi đây là tấn công tiêm lệnh gián tiếp (indirect prompt injection): kẻ tấn công không cần trò chuyện trực tiếp với mô hình mà chỉ cần chèn lệnh vào nguồn dữ liệu ứng dụng sẽ đọc, làm xóa nhòa ranh giới giữa dữ liệu và lệnh điều khiển. Bộ khung đánh giá AgentDojo [1] đo đồng thời khả năng hoàn thành công việc của người dùng và tỉ lệ bị dẫn dụ bởi các lệnh tiêm nhiễm đặt trong kết quả công cụ, qua đó cho thấy các biện pháp phòng vệ chỉ dựa trên lời nhắc trong prompt chưa đủ để bảo đảm an toàn. Chuẩn đánh giá τ-bench [10] đề xuất kiểm định agent dùng công cụ thông

qua trạng thái cuối cùng của cơ sở dữ liệu thay vì chỉ chấm điểm câu trả lời, đồng thời đo độ ổn định qua nhiều lượt chạy lặp.

Từ các kết quả trên, chúng tôi đặt ranh giới an toàn ở tầng kiến trúc thay vì chỉ dặn mô hình trong prompt. Bình luận và tiêu đề issue vẫn được bọc trong khối `<untrusted>` kèm chỉ dẫn không làm theo lệnh bên trong, nhưng chốt chặn chính nằm ở code thực thi: LLM không có quyền ghi, đầu ra bắt buộc khớp một schema đóng, mỗi giả thuyết phải trỏ tới mã sự kiện có thật qua bước kiểm tra của verifier, và mọi thao tác ghi vào Plane chỉ xảy ra khi người dùng bấm Duyệt. Chất lượng chẩn đoán được đo bằng độ khớp với nhãn của người chấm và tỉ lệ vượt qua verifier; độ trôi chảy của câu văn không được dùng làm thước đo.

2.4. Ước lượng công sức và lý do chọn cách tiếp cận cấp sprint

Một hướng tiếp cận quen thuộc để dự đoán trễ hạn là ước lượng trực tiếp công sức (story point) của từng issue. Tuy nhiên, nghiên cứu tái lập thực nghiệm về ước lượng story point bằng học sâu (Deep-SE) [7] cho thấy mô hình học sâu chỉ vượt qua mức tham chiếu trung vị một cách có ý nghĩa thống kê ở rất ít trường hợp, đồng thời kết luận độ tương đồng ngữ nghĩa của văn bản mô tả chưa đủ để phân biệt story point. Ở giai đoạn trước của đề tài (thực nghiệm E6), chúng tôi cũng đã thử ước lượng story point của issue mới từ 20 issue giống nhất đã hoàn thành trước đó trong cùng dự án. Trên 5 dự án, sai số MAE của phương pháp này là 1,61 so với 1,71 của trung vị dự án, với khoảng tin cậy 95% của hiệu số là $[-0{,}24;\ +0{,}01]$ chứa giá trị 0. Theo quy tắc kết luận đã khóa trước khi chạy, phương pháp này không hơn trung vị dự án. Do đó, chúng tôi không ước lượng thời gian hay story point cho từng issue riêng lẻ, mà xây dựng kế hoạch cấp sprint (mục 4.7) dựa trên tỉ lệ issue còn mở không kịp hoàn thành ở các sprint trước của cùng dự án.

CHƯƠNG 3. BÀI TOÁN VÀ DỮ LIỆU

3.1. Định nghĩa bài toán

Đơn vị phân tích của đề tài là một cặp issue-sprint: một issue xuất hiện trong hai sprint được tính là hai cặp riêng biệt. Mỗi sprint có thời điểm bắt đầu và thời điểm kết thúc; mốc quan sát t được chuẩn hóa theo phần trăm thời lượng sprint đã trôi qua, nhận bốn giá trị 0%, 25%, 50% và 75%. Tập cam kết của sprint gồm các issue thuộc sprint tại thời điểm bắt đầu. Một cặp issue-sprint mang nhãn "không xong" ($y = 1$) khi trạng thái cuối cùng quan sát được tại ngày kết thúc sprint không thuộc nhóm trạng thái hoàn thành (done, resolved, closed, complete). Các trạng thái mơ hồ như "deployed" hay "released to cloud" được loại khỏi tập gán nhãn.

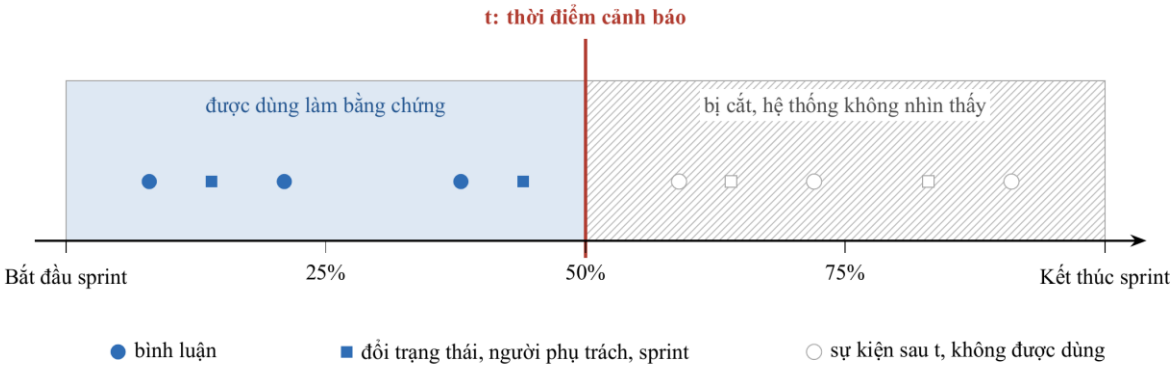
Tại mốc t, khối phát hiện chấm điểm rủi ro $p(t) \in [0, 1]$ cho mỗi issue chưa hoàn thành và chưa bị hủy (các issue mang trạng thái "invalid" hoặc "won't fix" không nằm trong diện cảnh báo). Trong một sprint có n issue đủ điều kiện xét duyệt tại mốc t, hệ thống chọn tối đa K cảnh báo theo công thức ngân sách:

$$K = \lceil 0{,}2n \rceil$$

Các issue được sắp xếp giảm dần theo điểm $p(t)$ và lấy K vị trí đứng đầu. Gọi P là tổng số issue thực sự không xong trong n issue đủ điều kiện (tỉ lệ nền $\pi = \frac{P}{n}$) và TP_K là số issue thực sự không xong nằm trong K cảnh báo được phát ra. Ba chỉ số đánh giá chất lượng cảnh báo tại mốc t là:

$$Precision_K = \frac{TP_K}{K}, \quad Recall_K = \frac{TP_K}{P}, \quad Lift_K = \frac{Precision_K}{\pi}$$

Với mỗi cảnh báo, khối chẩn đoán nhận tập sự kiện $E(t)$ gồm các bản ghi có mã sự kiện (`event_id`), loại sự kiện, dấu thời gian không muộn hơn t và nội dung văn bản; đầu ra là danh sách tối đa ba giả thuyết, mỗi giả thuyết gồm một nguyên nhân trong tập đóng và tập mã sự kiện trích xuất từ $E(t)$ làm bằng chứng.



Hình 3.1. Quy tắc cắt tại mốc t (minh họa ở mốc 50% thời lượng sprint): chỉ các sự kiện nằm bên trái mốc t được đưa vào mô hình và bộ thu thập

3.2. Dữ liệu TAWOS

Bản TAWOS 1.1 [6] sau khi nạp vào PostgreSQL gồm 39 dự án, 4.594 sprint, 458.232 issue và 9.253.419 dòng nhật ký thay đổi. Bảng 3.1 tóm tắt các bước lọc dữ liệu trước khi đưa vào đánh giá khối phát hiện.

Bảng 3.1. Các bước lọc từ dữ liệu gốc đến tập đánh giá khối phát hiện

Bước	Số lượng
Cặp issue-sprint có thông tin thành viên sprint	28.746
Gán nhãn kết quả rõ ràng	22.147 (77,04%)
Tổng số điểm cắt thời gian (cặp issue-sprint nhân 4 mốc)	88.588
Tập đánh giá cuối cùng sau tiêu chí chọn dự án	14 dự án, 1.962 sprint, 18.322 cặp

Tiêu chí chọn dự án được chúng tôi khóa cố định trước khi huấn luyện: mỗi dự án phải có ít nhất 50 sprint, 200 cặp issue-sprint và tối thiểu 30 cặp ở mỗi lớp nhãn. Độ chính xác của bước gán nhãn được kiểm tra chéo bằng một truy vấn SQL độc lập trên 765 cặp mẫu. Do mã người phụ trách trong TAWOS đã được ẩn danh, chúng tôi giữ nguyên định danh ẩn danh này.

Phần chẩn đoán, lập kế hoạch sprint và giao diện được triển khai và đo đạc trên dự án IS (Indy SDK của Hyperledger). Khi nhập vào Plane dưới mã dự án ISP để chạy giao diện, mỗi issue được đánh số thứ tự mới theo quy ước của Plane (ví dụ issue `IS-1119` của TAWOS mang mã `ISP-401` trên Plane).

3.3. Quy tắc cắt tại mốc t

Trong cơ sở dữ liệu TAWOS, các cột `assignee_id`, `priority`, `status` và `sprint_id` trên bảng `issue` lưu giá trị cuối cùng tại thời điểm thu thập bộ dữ liệu [6, 9]. Nếu đọc trực tiếp các cột này khi tái dựng trạng thái tại mốc t trong quá khứ, cả mô hình học máy lẫn LLM đều sẽ nhìn thấy thông tin tương lai. Do đó, chúng tôi tái dựng trạng

thái của từng issue tại mốc t từ bảng nhật ký thay đổi (`change_log`): lấy giá trị cập nhật cuối cùng của mỗi trường trong tập các bản ghi có dấu thời gian không vượt quá t (chuẩn hóa tên trường về chữ thường do dữ liệu gốc ghi lẫn lộn `"Sprint"` và `"sprint"`). Toàn bộ sự kiện cung cấp cho LLM, đặc trưng đầu vào của CatBoost và số liệu hiển thị trên thẻ cảnh báo đều tuân theo quy tắc cắt tại mốc t này và có bài kiểm thử tự động xác nhận không lọt sự kiện sau mốc t.

3.4. Các giả định về dữ liệu

Việc tái dựng lịch sử dựa trên ba giả định về cấu trúc dữ liệu (hệ quả đối với kết quả thực nghiệm được phân tích ở chương 9). Thứ nhất, nhật ký thay đổi được giả định là ghi nhận đủ các lần đổi trạng thái, người phụ trách và sprint; trong thực tế có khoảng 5-7% trường hợp trạng thái hoặc người phụ trách tái dựng tại mốc t lệch từ vựng với cột cuối cùng. Thứ hai, danh sách thành viên của sprint tại mốc t được xấp xỉ qua cột `sprint_id` cuối cùng kết hợp điều kiện thời điểm tạo issue không muộn hơn t. Thứ ba, các liên kết phụ thuộc ("chặn") giữa hai issue được xem là đã hình thành trước mốc t nếu cả hai issue đều được tạo trước mốc t, bởi bảng `issue_link` trong TAWOS không có cột thời điểm tạo liên kết.

CHƯƠNG 4. PHƯƠNG PHÁP

Chương này trình bày phương pháp của từng thành phần theo trình tự xử lý dữ liệu: phát hiện rủi ro (mục 4.1), bộ thu thập sự kiện bằng SQL (mục 4.2), chẩn đoán bằng LLM kèm verifier (mục 4.3, 4.4), hai ghi chú theo luật (mục 4.5), thực thi hành động (mục 4.6), lập kế hoạch cấp sprint (mục 4.7), duyệt theo lô (mục 4.8), agent hỏi đáp (mục 4.9) và sổ cảnh báo (mục 4.10). Mục 4.11 tổng hợp bảy bất biến an toàn của hệ thống.

4.1. Khối phát hiện rủi ro

Khối phát hiện kế thừa từ giai đoạn thực nghiệm E1-E9 và được giữ cố định trong giai đoạn này. Tại mỗi mốc t, mỗi cặp issue-sprint được biểu diễn bằng năm nhóm đặc trưng dựa trên tiến độ công việc đã diễn ra trước đó: nhóm thời gian (độ dài sprint, số ngày đã trôi qua, số ngày còn lại), nhóm trạng thái thực thi (trạng thái hiện tại, cờ hoàn thành, số ngày liên tiếp không có hoạt động), nhóm cường độ hoạt động trước mốc t (tổng số sự kiện, số lần đổi trạng thái, đổi người phụ trách, đổi ước lượng), nhóm thuộc tính tĩnh (loại issue, độ ưu tiên, điểm ước lượng, tuổi của issue) và nhóm bối cảnh sprint (tổng số issue, tỉ lệ issue cùng sprint đã hoàn thành). Mô hình tĩnh chỉ sử dụng thông tin có sẵn ở thời điểm bắt đầu sprint ($t = 0\%$), còn mô hình động kết hợp thêm lịch sử hoạt động tính đến mốc t. Cả hai sử dụng thuật toán CatBoost [5] với bộ siêu tham số khóa trước khi huấn luyện: 400 cây quyết định, độ sâu 5, tốc độ học 0,05 và hệ số điều chuẩn L2 bằng 5. Điểm đầu ra $z(x)$ của mô hình được hiệu chuẩn thành xác suất qua hàm logistic (Platt scaling) khớp trên tập validation:

$$P(y = 1 \mid x) = \frac{1}{1 + exp(-(a \cdot z(x) + b))}$$

Chúng tôi so sánh mô hình với hai mức tham chiếu: tỉ lệ nền (chọn ngẫu nhiên trong tập issue đủ điều kiện) và một quy tắc sắp xếp đơn giản (gán điểm thấp nhất cho issue đã xong, còn với các issue đang mở thì xếp theo số ngày không có hoạt động giảm dần).

Mô hình được kiểm định trên hai kịch bản chia dữ liệu độc lập. Kịch bản chia theo thời gian huấn luyện trên các sprint diễn ra trước và kiểm tra trên các sprint diễn ra sau của cùng dự án. Kịch bản chia theo dự án giữ lại lần lượt từng dự án trong 14 dự án làm tập kiểm tra và huấn luyện trên 13 dự án còn lại. Mọi mốc t của cùng một cặp issue-sprint luôn nằm trong cùng một tập chia. Khi ước lượng khoảng tin cậy 95% của chênh lệch hiệu năng, chúng tôi lấy mẫu bootstrap ghép cặp 2.000 lần theo cụm sprint để tính đến tương quan giữa các issue trong cùng một sprint.

4.2. Bộ thu thập sự kiện bằng năm truy vấn SQL

Bộ thu thập gồm năm truy vấn SQL cố định, không dùng AI; mỗi truy vấn nhận mã issue cùng mốc t và trả về danh sách sự kiện chuẩn hóa. Mỗi sự kiện gồm mã sự kiện (`event_id`), loại sự kiện, dấu thời gian và một dòng mô tả. LLM ở bước sau chỉ được phép trích dẫn các mã sự kiện có trong danh sách này.

Năm truy vấn trong bộ thu thập lấy dữ liệu từ năm nguồn:

1. Thảo luận: tối đa 8 bình luận gần nhất có thời điểm không muộn hơn t (cắt tối đa 400 ký tự mỗi bình luận).
2. Hoạt động của người phụ trách: xác định người phụ trách tại mốc t từ nhật ký thay đổi, sau đó lấy tối đa 5 lần người này cập nhật trạng thái trên các issue khác trong 7 ngày trước mốc t (dùng làm bằng chứng cho nguyên nhân "người phụ trách bị kéo sang việc khác").
3. Chuỗi chặn: lần theo quan hệ phụ thuộc "issue A chờ issue B" đến độ sâu 2 và lấy trạng thái tại mốc t của từng issue chặn. Khi xây dựng bảng ánh xạ loại liên kết, chúng tôi chỉ giữ các liên kết có hướng rõ ràng; 446 liên kết `"is dependent of"` và 52 liên kết `"requires"` bị loại vì thứ tự tạo giữa hai đầu liên kết chia xấp xỉ 50/50, không xác định được chiều phụ thuộc.
4. Tình trạng sprint: phần trăm thời lượng sprint đã trôi qua, số issue đã xong và số issue còn mở tại mốc t.
5. Lịch sử issue: tối đa 8 lần đổi trạng thái, mức ưu tiên, người phụ trách, sprint hoặc story point trước mốc t, kèm một sự kiện tổng hợp ghi nhận tổng số lần issue được đưa vào sprint nếu từng bị dồn qua nhiều sprint.

Với những sự kiện có nhắc tới một issue khác (như issue mà người phụ trách đang làm hoặc issue đang chặn), bộ thu thập lưu kèm mã dự án của issue đó và một dòng mô tả đã ẩn tiêu đề. Dịch vụ backend sẽ hiển thị dòng ẩn tiêu đề này nếu người đang xem không có quyền truy cập vào dự án của issue liên quan (mục 4.11).

4.3. Chẩn đoán nguyên nhân bằng một lần gọi LLM

Với mỗi cảnh báo, hệ thống gọi LLM một lần. Prompt hệ thống (phụ lục A) yêu cầu mô hình trả về đối tượng JSON gồm danh sách giả thuyết, thông tin còn thiếu (`missing_info`), một hành động đề xuất (`action`) và câu nhắc (`nudge_text`). Mỗi giả thuyết chọn một nguyên nhân trong tập đóng 6 nhãn, liệt kê các mã sự kiện làm bằng chứng và nêu một câu giải thích; mỗi cảnh báo giữ tối đa 3 giả thuyết xếp theo độ mạnh giảm dần. Sáu nhãn nguyên nhân gồm: người phụ trách bị kéo sang việc khác

(pulled_away), bị chặn bởi issue khác (blocked), đang chờ phản hồi từ một người (waiting_person), chờ quyết định kỹ thuật hoặc sản phẩm (needs_decision), phạm vi yêu cầu chưa rõ (unclear_scope) và bị dồn từ sprint trước (carry_over). Nếu các sự kiện thu thập được không đủ để kết luận nguyên nhân nào, mô hình trả về danh sách giả thuyết rỗng kèm gợi ý cần hỏi thêm thông tin gì; thẻ cảnh báo khi đó hiển thị trạng thái "chưa rõ".

Tiêu đề issue và nội dung sự kiện được bọc trong cặp thẻ <untrusted> (mọi chuỗi con "untrusted" có sẵn trong văn bản gốc đều bị xóa trước khi ghép vào prompt để dữ liệu người dùng không thể tự đóng thẻ). Đầu ra của LLM được kiểm tra lược đồ bằng Pydantic; nếu sai định dạng JSON hoặc gặp lỗi mạng, thẻ cảnh báo được chuyển về trạng thái "chưa rõ" không kèm hành động để không làm dừng tiến trình chạy theo lô. Thực nghiệm trong báo cáo sử dụng mô hình openai/gpt-6-luna qua cổng OpenRouter có khóa trần chi phí trên từng yêu cầu.

4.4. Bước hậu kiểm bằng code (Verifier)

Verifier là bước kiểm tra bằng code chạy ngay sau khi nhận phản hồi từ LLM và trước khi lưu thẻ cảnh báo. Một giả thuyết bị verifier loại bỏ nếu: trích dẫn mã sự kiện không có trong danh sách sự kiện của bộ thu thập, lặp lại nhãn nguyên nhân của giả thuyết đứng trước, hoặc trích dẫn loại sự kiện không khớp với nhãn nguyên nhân. Cụ thể, pulled_away phải dẫn chiếu ít nhất một sự kiện hoạt động của người phụ trách trên issue khác; blocked phải dẫn chiếu ít nhất một sự kiện chuỗi chặn đang mở; carry_over phải dẫn chiếu sự kiện đếm số lần đưa vào sprint; ba nhãn còn lại phải dẫn chiếu ít nhất một bình luận. Sau đợt đánh giá đầu tiên của module M13 (mục 7.2), chúng tôi bổ sung quy tắc đẩy giả thuyết carry_over xuống vị trí cuối cùng khi thẻ có nhiều hơn một giả thuyết hợp lệ, vì việc bị dồn từ sprint trước chỉ là dấu hiệu bề mặt khi đã có một nguyên nhân cụ thể hơn.

Verifier đồng thời kiểm tra câu nhắc gửi cho người phụ trách (nudge_text). Câu nhắc bị loại bỏ nếu không chứa đúng mã issue đang xét, dài dưới 20 hoặc trên 600 ký tự, chứa liên kết web, ký tự @ hoặc thẻ HTML, nêu số ngày ngừng hoạt động lệch quá 1 ngày so với thực tế, nhắc tới mã issue của dự án khác, hoặc khi issue vừa có hoạt động trong vòng 2 ngày gần nhất. Tương tự, nếu LLM đề xuất chuyển sang sprint kế tiếp nhưng dự án không có sprint kế tiếp đang mở thì đề xuất hành động đó cũng bị hủy.

4.5. Hai ghi chú bối cảnh theo luật

Bên cạnh các giả thuyết nguyên nhân của từng issue, hệ thống kiểm tra thêm hai ghi chú bối cảnh bằng luật tất định và hiển thị riêng trên thẻ cảnh báo. Ghi chú "sprint

quá tải" (`sprint_overload`) bật khi sprint đã đi qua ít nhất 50% thời lượng, có từ 5 issue trở lên và tỉ lệ issue còn mở từ 60% trở lên. Ghi chú "có thể đã hoàn thành nhưng chưa cập nhật trạng thái" (`done_not_updated`) bật khi có bình luận trong 3 ngày trước mốc t chứa từ khóa báo hoàn tất (`done`, `fixed`, `merged`, `completed`, `resolved`, `deployed`, `released`) nhưng trạng thái của issue vẫn chưa chuyển sang hoàn thành; nội dung trên thẻ luôn giữ từ "có thể" kèm trích dẫn bình luận đó. Các ngưỡng này được đặt theo quan sát ban đầu và chưa tinh chỉnh trên tập nhãn riêng.

4.6. Thực thi hành động và cơ chế Duyệt / Bỏ qua / Hoàn tác

Tập hành động có thể thực thi tự động lên Plane được giới hạn ở ba thao tác: nâng mức ưu tiên của issue lên High (`raise_priority`), đăng bình luận nhắc người phụ trách (`nudge_owner`), và chuyển issue sang sprint kế tiếp (`move_next_cycle`, trong giao diện Plane gọi là cycle kế tiếp). Trước tiên, quy tắc code gán hành động mặc định: nếu mốc cảnh báo từ 75% trở đi và có sprint kế tiếp chưa đóng thì chọn chuyển sang sprint sau; nếu issue đã ngừng hoạt động từ 3 ngày trở lên thì chọn đăng lời nhắc theo mẫu câu chuẩn; các trường hợp còn lại chọn nâng mức ưu tiên. Đầu ra của LLM chỉ được phép thay mẫu câu nhắc bằng câu nhắc do mô hình viết (nếu qua được verifier) hoặc đổi sang nâng mức ưu tiên, nhưng không được ghi đè quyết định chuyển sprint của quy tắc code. Các đề xuất như phân công lại người làm hay tách nhỏ issue chỉ hiển thị dưới dạng văn bản gợi ý trên thẻ.

Trên giao diện Plane, mỗi thẻ cảnh báo có các nút Duyệt, Bỏ qua và Hoàn tác. Khi người dùng bấm Duyệt, dịch vụ risk xác thực phiên đăng nhập Plane của người dùng đó, gọi API của Plane để thực hiện thay đổi, lưu lại giá trị trước khi sửa (mức ưu tiên cũ, mã bình luận vừa đăng hoặc sprint cũ) và ghi nhận ai đã duyệt. Lời gọi sang API của Plane luôn chạy trước bước cập nhật trạng thái cảnh báo trong cơ sở dữ liệu risk để nếu API gặp lỗi thì thẻ vẫn ở trạng thái chờ duyệt. Khi bấm Hoàn tác, hệ thống khôi phục lại giá trị cũ đã lưu. Khi bấm Bỏ qua, hệ thống chỉ đóng cảnh báo mà không sửa dữ liệu trên Plane.

4.7. Lập kế hoạch cấp sprint (Module M14)

Khối lập kế hoạch cấp sprint trả lời hai câu hỏi ở cấp toàn sprint: trong số các issue còn mở tại mốc t, số không xong dự kiến là bao nhiêu, và nên đưa những issue nào vào tập chuyển sang sprint kế tiếp? Cả hai bước này đều do thuật toán tính toán; LLM không tham gia ước lượng số lượng hay chọn issue.

Gọi $S_{trước}$ là tập các sprint đã kết thúc trước thời điểm hiện tại trong cùng dự án. Với mỗi sprint lịch sử $s \in S_{trước}$, gọi $O_s(t)$ là số issue còn mở tại cùng mốc phần trăm

thời gian t và $U_s(t)$ là số lượng trong số đó không hoàn thành khi sprint s kết thúc. Tỉ lệ không hoàn thành lịch sử r tại mốc t và số không xong dự kiến b của sprint hiện tại (đang có $O(t)$ issue còn mở) được tính theo công thức:

$$r = \frac{\sum_{s \in S_{trước}} U_s(t)}{\sum_{s \in S_{trước}} O_s(t)}, \quad b = round(r \cdot O(t))$$

Nếu dự án có ít hơn 3 sprint lịch sử hoặc tổng số issue đang mở qua các sprint trước nhỏ hơn 15, hệ thống không sinh kế hoạch tự động do cỡ mẫu quá nhỏ. Khi $b > 0$, thuật toán chọn b issue đang mở có điểm rủi ro $p(t)$ cao nhất đưa vào tập chuyển. Tiếp theo, tập chuyển được đóng bắc cầu theo quan hệ phụ thuộc chờ-nhau: nếu issue A đang chờ issue B mà B nằm trong tập chuyển, thì A cũng được đưa vào tập chuyển (với nhãn lý do là "đi cùng" - `dependent`), vì A không thể hoàn thành khi issue chặn nó đã dời sang sprint sau. Nhờ bước đóng bắc cầu này, tổng số issue trong tập chuyển có thể lớn hơn b. Lý do hiển thị trên từng dòng issue được lấy từ thẻ cảnh báo đã qua verifier của chính issue đó, còn câu tóm tắt đầu kế hoạch được điền từ mẫu câu cố định.

Trong môi trường vận hành thực tế, kế hoạch sẽ chuyển issue từ sprint đang chạy sang sprint kế tiếp. Do toàn bộ các sprint trong bộ dữ liệu lịch sử TAWOS đều đã kết thúc từ nhiều năm trước và Plane không cho phép chuyển issue vào một sprint đã đóng, chúng tôi xây dựng thêm chế độ phát lại (`--demo`): tạo hai sprint đang mở trên Plane và nạp các issue còn mở của một sprint lịch sử vào sprint đầu để kiểm tra luồng duyệt và hoàn tác trên giao diện.

4.8. Duyệt và hoàn tác theo lô (Module M15)

Trên thẻ kế hoạch sprint, người quản lý có thể tích chọn hoặc bỏ chọn từng dòng issue rồi bấm Duyệt một lần cho cả tập chuyển. Ở tầng backend, mỗi dòng được chọn vẫn đi qua hàm thực thi hành động đóng của mục 4.6 (`move_next_cycle`), nên thao tác theo lô không mở thêm quyền ghi ngoại lệ nào. Kết quả của từng dòng được lưu độc lập: nếu một issue bị API từ chối chuyển sprint, lỗi của dòng đó được ghi riêng mà không làm hỏng các dòng hợp lệ còn lại. Khi bấm Hoàn tác kế hoạch, hệ thống đọc lại sprint gốc đã lưu của từng dòng đã chuyển thành công và đưa các issue đó về sprint cũ.

4.9. Agent hỏi đáp chỉ đọc (Module M16)

Agent hỏi đáp cho phép người quản lý tra cứu tình hình rủi ro bằng ngôn ngữ tự nhiên (tiếng Việt hoặc tiếng Anh), ví dụ "những issue nào đang có nguy cơ cao nhất và vì sao?". Chúng tôi cài đặt trực tiếp vòng lặp gọi công cụ (tool-calling loop) trên client LLM: mô hình chọn tên công cụ và tham số, tầng code thực thi hàm chỉ đọc và trả kết quả về cho mô hình cho đến khi có câu trả lời cuối cùng. Năm công cụ cung cấp cho

agent đều là hàm có tham số định kiểu chặt, chỉ đọc dữ liệu và lọc quyền truy cập dự án ngay trong câu truy vấn. Năm công cụ gồm: liệt kê các cảnh báo có điểm rủi ro cao nhất (`list_at_risk`), giải thích chi tiết một cảnh báo theo mã issue (`explain_alert`), tổng hợp thống kê nguyên nhân và quyết định (`decision_summary`), tra cứu kế hoạch cấp sprint (`sprint_plans`) và tìm kiếm cảnh báo theo từ khóa (`search_alerts`).

Để kiểm soát an toàn và chi phí, vòng lặp của agent hỏi đáp có ba chốt chặn bằng code:

1. Giới hạn tối đa 5 vòng gọi công cụ cho mỗi câu hỏi và cắt kết quả trả về của mỗi lần gọi ở 4.000 ký tự.
2. Sau khi mô hình sinh câu trả lời, một bộ kiểm tra quét mọi mã issue xuất hiện trong văn bản: nếu có mã issue nào không nằm trong kết quả công cụ vừa truy xuất (và không phải mã do chính người dùng gõ trong câu hỏi), câu trả lời bị hủy và thay bằng thông báo từ chối chuẩn.
3. Mọi dữ liệu văn bản lấy từ cơ sở dữ liệu đều được bọc trong khối `<untrusted>`. Chúng tôi không dùng kỹ thuật sinh truy vấn SQL tự do (text-to-SQL), vì với text-to-SQL thì việc lọc quyền xem theo dự án phụ thuộc vào câu truy vấn do LLM tự viết, dễ dẫn đến lộ dữ liệu chéo dự án.

4.10. Sổ cảnh báo theo dõi vòng đời (Module M17)

Sổ cảnh báo lưu lịch sử xuất hiện của từng cảnh báo qua các lần hệ thống chạy chấm điểm định kỳ. Trên mỗi bản ghi cảnh báo, hệ thống lưu số lần cảnh báo đã được kiểm tra (`seen_count`) và điểm rủi ro ở lần chạy liền trước (`prev_score`), hiển thị trên thẻ cảnh báo dòng trạng thái `"Checked N times. Risk score X in the run before, Y now"`. Nếu một cảnh báo từng bị người dùng bấm Bỏ qua nhưng ở lần chạy sau điểm rủi ro tăng thêm tối thiểu 0,15:

$$p_{mới} - p_{trước} \geq 0{,}15$$

thì hệ thống tự động mở lại cảnh báo đó về trạng thái chờ duyệt (ngưỡng 0,15 hiện được đặt theo quan sát). Những cảnh báo đã được bấm Duyệt thì không tự mở lại. Khi hệ thống tự mở lại một cảnh báo đã bỏ qua, bản ghi kiểm toán ghi nhận tác nhân là dịch vụ hệ thống (`risk service`) để phân biệt với thao tác Hoàn tác của người dùng.

4.11. Bảy bất biến an toàn của hệ thống

Toàn bộ các thành phần từ bước tiền xử lý dữ liệu đến giao diện người dùng đều tuân theo bảy bất biến kiến trúc trong bảng 4.1. Mỗi bất biến đều được kiểm tra bằng code và có bài kiểm thử tự động đi kèm.

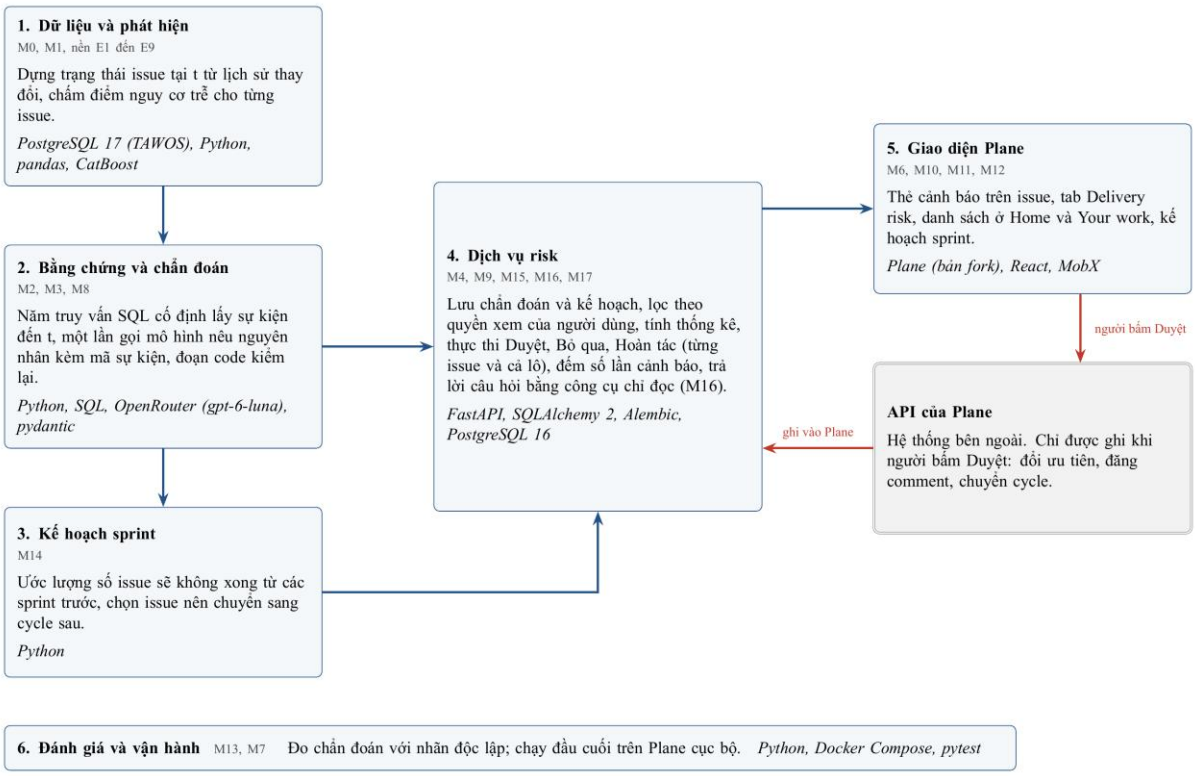
Bảng 4.1. Bảy bất biến an toàn và nơi bảo đảm trong hệ thống

Bất biến	Nội dung ràng buộc	Nơi bảo đảm và kiểm thử
Cắt tại mốc t	Mọi bằng chứng và đặc trưng chỉ dùng sự kiện xảy ra không muộn hơn t; trạng thái issue tái dựng từ nhật ký thay đổi	Bộ thu thập (4.2), kiểm thử tự động xác nhận không có sự kiện sau t
Bám bằng chứng	Mỗi giả thuyết trên thẻ cảnh báo phải trỏ tới mã sự kiện có thật; giả thuyết không qua hậu kiểm bị loại bỏ	Verifier (4.4)
LLM không có quyền ghi	Dữ liệu trong Plane chỉ thay đổi khi người dùng bấm Duyệt; mọi hành động ngoài tập đóng bị từ chối	Dịch vụ backend và lược đồ dữ liệu (4.6)
Văn bản người dùng là dữ liệu	Bình luận và tiêu đề được bọc trong khối `<untrusted>`, giới hạn độ dài, không thực thi lệnh bên trong	Khối chẩn đoán (4.3) và agent hỏi đáp (4.9)
Tôn trọng quyền xem	Bằng chứng liên quan tới issue khác chỉ hiện tiêu đề khi người xem có quyền truy cập dự án đó; thống kê chỉ đếm trên dự án được phép xem	Dịch vụ backend, các công cụ của agent hỏi đáp (4.9)
Số liệu định lượng do code tính	Số không xong dự kiến và tập chuyển do thuật toán tính; LLM không tự sinh con số ngoài kế hoạch	Khối kế hoạch sprint (4.7)
Xử lý theo lô qua tập hành động đóng	Mỗi dòng trong lô phê duyệt đi qua đúng hành động đóng và hoàn tác độc lập; agent hỏi đáp chỉ có công cụ đọc	Khối duyệt theo lô (4.8) và agent hỏi đáp (4.9)

CHƯƠNG 5. HIỆN THỰC HỆ THỐNG

5.1. Kiến trúc hệ thống

Hệ thống gồm sáu khối chức năng như minh họa trên hình 5.1. Luồng dữ liệu đi theo một chiều: khối 1 cắt dữ liệu TAWOS tại mốc t và chấm điểm rủi ro; khối 2 dùng bộ thu thập lấy sự kiện lịch sử và chạy chẩn đoán; khối 3 tính kế hoạch cấp sprint; khối 4 lưu trữ kết quả, lọc quyền truy cập theo người dùng và điều phối các thao tác; khối 5 hiển thị trên giao diện Plane. Khối 6 đảm nhiệm việc kiểm định độc lập và đóng gói môi trường chạy. Hệ thống Plane là ứng dụng đích bên ngoài và chỉ nhận lệnh ghi dữ liệu khi người dùng bấm nút Duyệt.



Hình 5.1. Sáu khối chức năng của hệ thống; mũi tên đỏ biểu thị luồng ghi duy nhất từ dịch vụ risk sang API của Plane khi người dùng bấm Duyệt

Bảng 5.1 tóm tắt công nghệ và chức năng của từng khối trong hệ thống.

Bảng 5.1. Các khối chức năng và công nghệ sử dụng trong hệ thống

Khối	Công nghệ	Chức năng chính
1. Dữ liệu và phát hiện	PostgreSQL 17 (TAWOS), Python, pandas, CatBoost	Tái dựng trạng thái issue tại mốc t từ nhật ký thay đổi và chấm điểm nguy cơ trễ hạn cho từng issue đang mở.
2. Bằng chứng và chẩn đoán	Python, SQL, OpenRouter (`openai/gpt-6-luna`), Pydantic	Trích xuất sự kiện đến mốc t bằng bộ thu thập, gọi LLM một lần để nêu giả thuyết nguyên nhân kèm mã sự kiện, kiểm tra qua verifier và đăng cảnh báo.

Khối	Công nghệ	Chức năng chính
3. Kế hoạch sprint	Python	Tính số không xong dự kiến từ lịch sử các sprint trước và chọn tập chuyển sang sprint kế tiếp.
4. Dịch vụ risk	FastAPI, SQLAlchemy 2, Alembic, PostgreSQL 16	Lưu trữ chẩn đoán và kế hoạch, lọc dữ liệu theo quyền xem, tổng hợp thống kê, thực thi Duyệt, Bỏ qua, Hoàn tác (đơn lẻ và theo lô), quản lý sổ cảnh báo và vận hành agent hỏi đáp chỉ đọc.
5. Giao diện Plane	Plane (bản fork), React, MobX	Hiển thị thẻ cảnh báo trên từng issue, tab phân tích Delivery risk, danh sách cảnh báo tại Home và Your work, thẻ kế hoạch sprint và khung hỏi đáp Ask.
6. Đánh giá và vận hành	Python, Docker Compose, pytest	Đo độ khớp của chẩn đoán với nhãn của người chấm và khởi chạy hệ thống đầu cuối trên môi trường cục bộ.

Khối 2 và khối 3 chạy dưới dạng tác vụ nền theo lô (batch jobs), tách khỏi luồng tải trang của người dùng vì mỗi cảnh báo cần một lượt gọi LLM. Ở khối 4, chúng tôi tách dịch vụ risk thành một backend độc lập với cơ sở dữ liệu riêng, buộc mọi thao tác thay đổi dữ liệu trên Plane phải đi qua API chính thức dưới quyền của phiên người dùng đang đăng nhập. Ở mỗi yêu cầu truy vấn, dịch vụ risk đều đối chiếu lại danh sách dự án mà Plane cho phép người dùng đó nhìn thấy. Khối 5 tích hợp thẻ cảnh báo ngay tại màn hình chi tiết issue của Plane và thêm một tab riêng mang tên Delivery risk trong mục Analytics.

5.2. Giao diện người dùng trên Plane

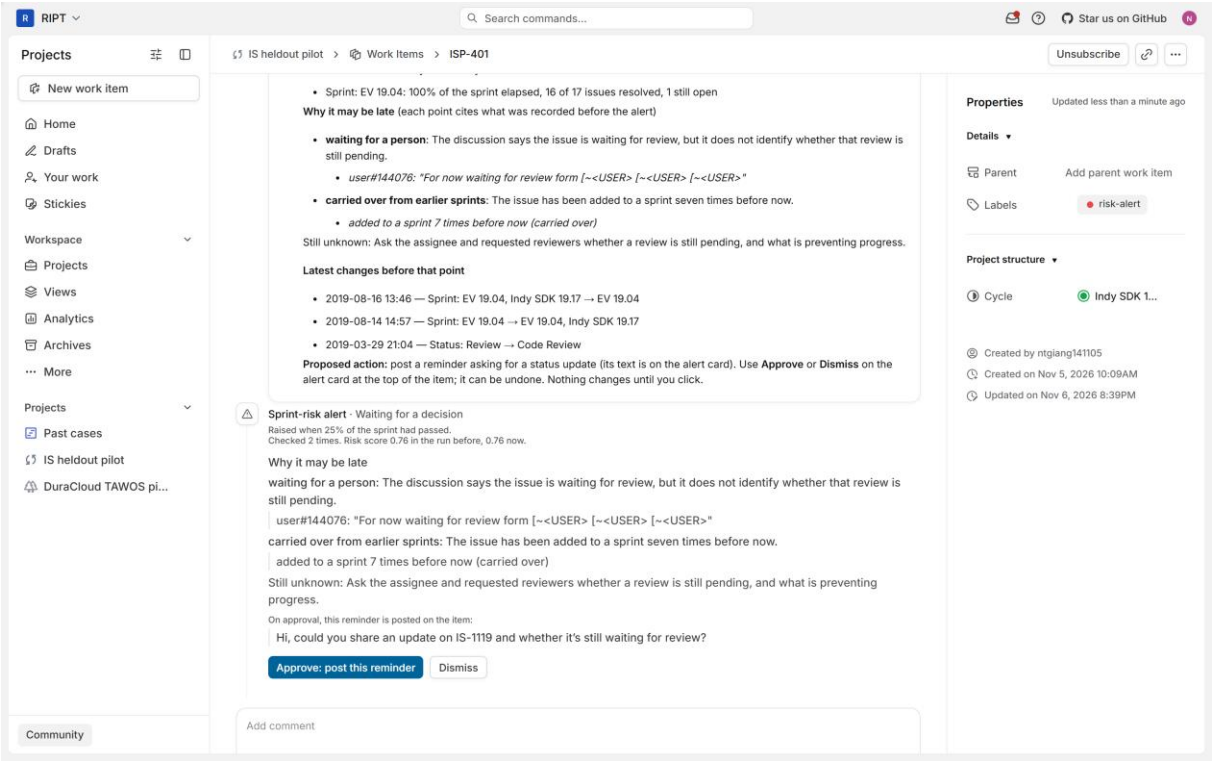
Các hình chụp màn hình trong mục này được lấy từ lượt chạy thứ hai của tiến trình đăng cảnh báo trên 81 cảnh báo của dự án IS, sau khi đã cập nhật quy tắc sắp xếp thứ tự nguyên nhân (mục 7.2).

Trên trang chi tiết của từng issue (hình 5.2), tiến trình nền đăng một bình luận tóm tắt gồm tình trạng sprint, các giả thuyết kèm sự kiện trích dẫn, thông tin còn thiếu, danh sách thay đổi gần nhất trước mốc t và hành động đề xuất. Ngay bên dưới là thẻ cảnh báo tương tác: hiển thị mốc t, dòng trạng thái từ sổ cảnh báo (`"Checked 2 times. Risk score 0.76 in the run before, 0.76 now"`), danh sách giả thuyết, câu hỏi cần làm rõ, câu nhắc dự kiến gửi đi nếu người dùng duyệt, cùng hai nút Duyệt và Bỏ qua (nút Hoàn tác xuất hiện sau khi người dùng đã bấm Duyệt).

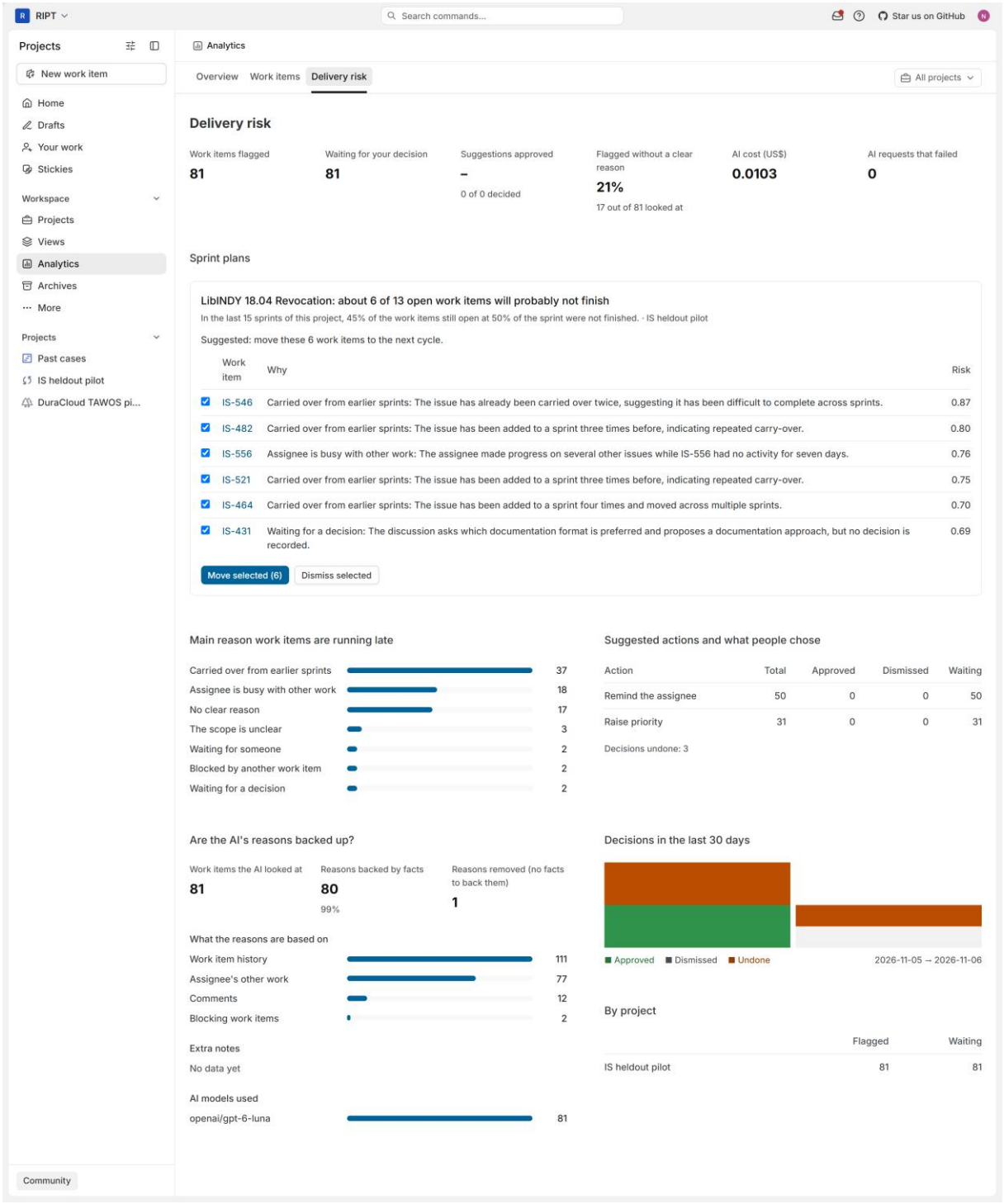
Trong mục Analytics, tab Delivery risk (hình 5.3) hiển thị thống kê cấp dự án: hàng chỉ số đầu trang (81 cảnh báo, 81 đang chờ quyết định, 17 trên 81 cảnh báo chưa rõ nguyên nhân, tổng chi phí 0,0103 USD, 0 lượt gọi lỗi), thẻ kế hoạch cấp sprint, khung

hỏi đáp với agent hỏi đáp, biểu đồ phân bố nguyên nhân đứng đầu, thống kê hành động đề xuất và quyết định phê duyệt, tỉ lệ bám bằng chứng (80 trên 81 thẻ có lý do được sự kiện lịch sử hỗ trợ, 1 giả thuyết bị verifier loại bỏ), lịch sử quyết định theo ngày và bảng tổng hợp theo từng dự án. Chỉ số `"Decisions undone: 3"` ghi nhận 3 thao tác hoàn tác thực hiện trong quá trình kiểm thử trước đó. Bộ lọc dự án ở góc trên của trang chỉ thu hẹp phạm vi hiển thị trong tập dự án mà tài khoản hiện tại có quyền xem.

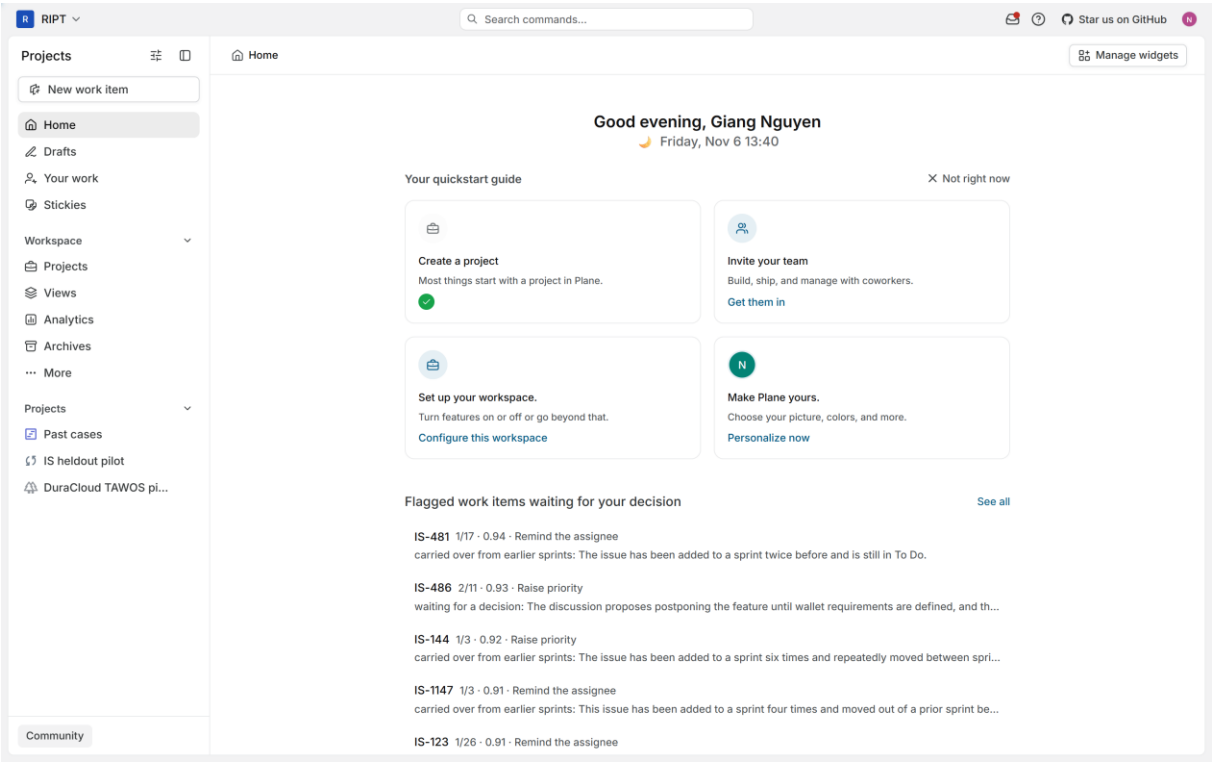
Tại trang chủ Home (hình 5.4), một bảng danh sách liệt kê các cảnh báo đang chờ xử lý kèm mã issue, thứ hạng rủi ro trong sprint, điểm số, hành động đề xuất và nguyên nhân đứng đầu. Trên trang cá nhân Your work (hình 5.5, nửa trên), danh sách hiện để trống do mã người phụ trách trong TAWOS đã bị ẩn danh hóa nên không khớp với tài khoản người dùng Plane (mục 9.2); trong khi đó, chế độ xem đã lưu (saved view) mang tên `"Flagged by the risk check"` (hình 5.5, nửa dưới) lọc theo nhãn `risk-alert` và hiển thị đủ 81 issue có cảnh báo của dự án ISP.



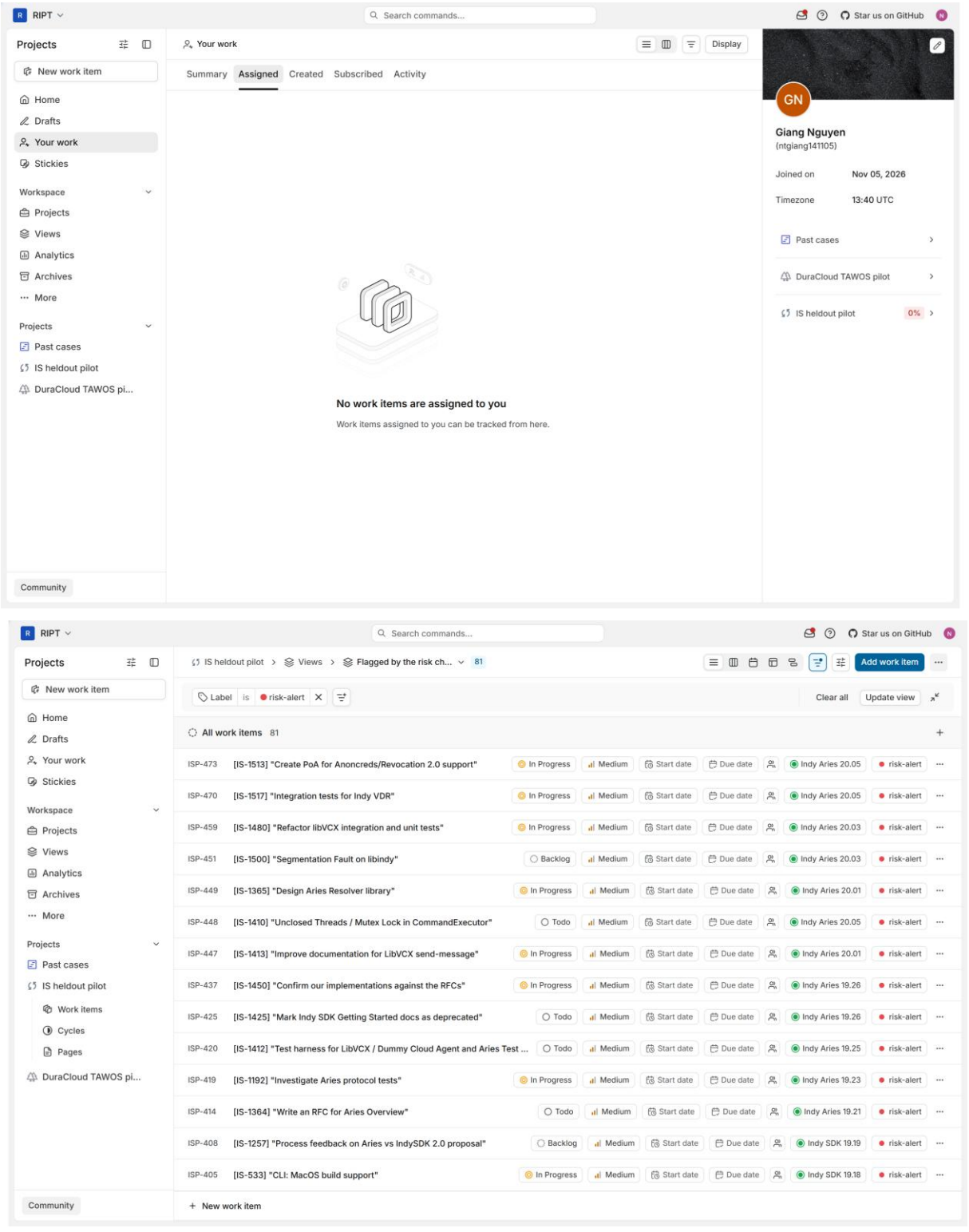
Hình 5.2. Thẻ cảnh báo trên trang chi tiết issue ISP-401 trong Plane (lượt chạy thứ hai); phía trên là bình luận tóm tắt do tiến trình tự động đăng



Hình 5.3. Màn hình Delivery risk trong mục Analytics (lượt chạy thứ hai), hiển thị thống kê dự án và thẻ kế hoạch sprint



Hình 5.4. Danh sách cảnh báo đang chờ phê duyệt trên trang chủ Home (lượt chạy thứ hai)

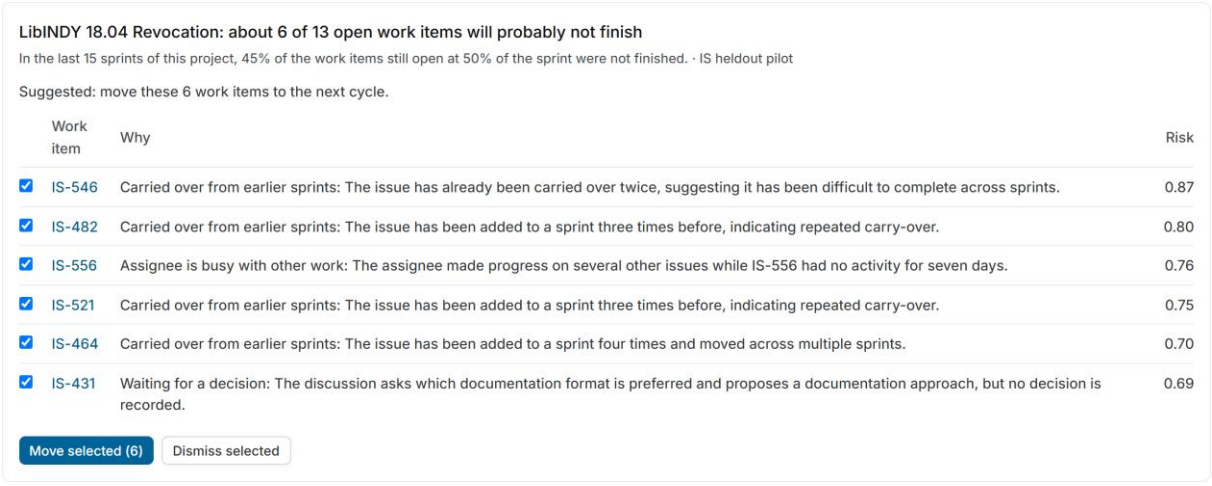


Hình 5.5. Màn hình Your work trống do dữ liệu ẩn danh (trên) và chế độ xem đã lưu "Flagged by the risk check" liệt kê 81 issue có cảnh báo (dưới)

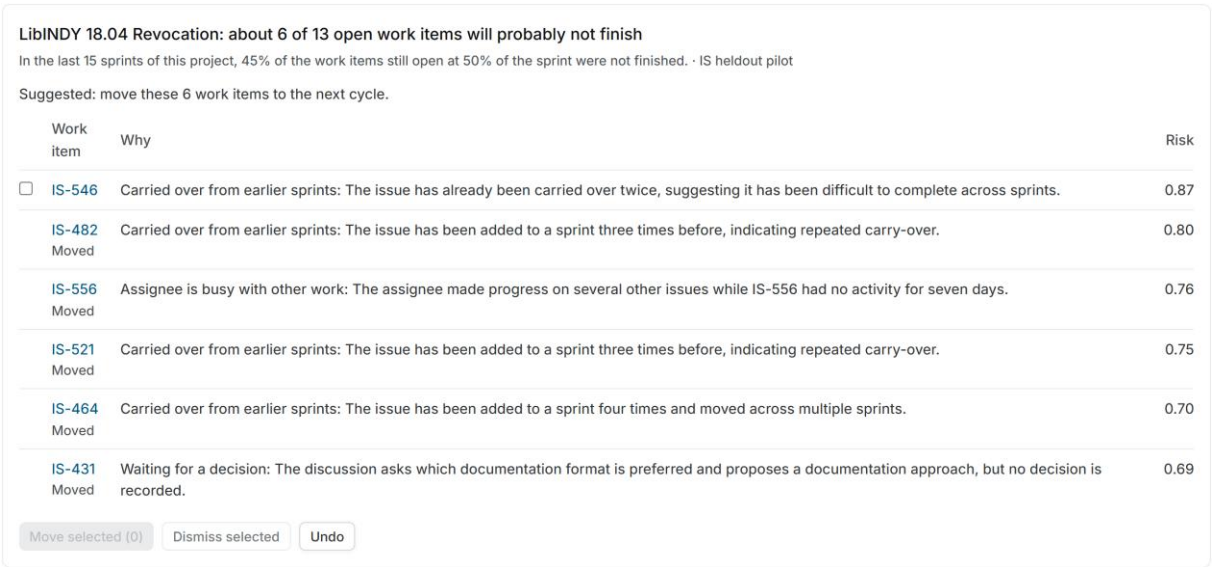
Thẻ kế hoạch cấp sprint (hình 5.6) nằm ngay trong tab Delivery risk. Kế hoạch minh họa trên hình thuộc về sprint `"LibINDY 18.04 Revocation"` tại mốc 50% thời lượng, chạy qua chế độ phát lại demo (mục 4.7): tại mốc này sprint còn 13 issue đang mở; thống kê trên 15 sprint liền trước của cùng dự án cho thấy 45% số issue còn mở ở mốc 50% đã không kịp hoàn thành, nên thuật toán tính số không xong dự kiến là $b = 6$ và đưa 6 issue có điểm rủi ro cao nhất (IS-546, IS-482, IS-556, IS-521, IS-464 và IS-

431) vào tập chuyển. Mỗi dòng hiển thị kèm nguyên nhân lấy từ thẻ cảnh báo đã qua verifier cùng điểm rủi ro; tổng chi phí gọi LLM cho kế hoạch này là 0,00074 USD. Khi kiểm thử duyệt theo lô (hình 5.7), chúng tôi bỏ chọn dòng IS-546 và bấm duyệt cho 5 dòng còn lại: kiểm tra qua API của Plane xác nhận cả 5 issue được chọn đã chuyển sang sprint kế tiếp, còn IS-546 giữ nguyên ở sprint cũ. Khi bấm nút Undo (Hoàn tác), cả 5 issue quay về sprint ban đầu và thẻ kế hoạch trở lại trạng thái chờ duyệt.

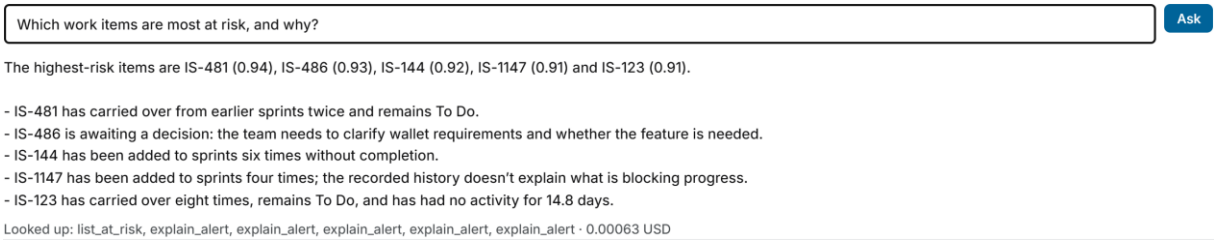
Ngay bên dưới thẻ kế hoạch trong tab Delivery risk là khung hỏi đáp `"Ask about delivery risk"` (hình 5.8). Người dùng nhập câu hỏi vào ô văn bản và bấm nút `"Ask"`; agent hỏi đáp gọi các công cụ chỉ đọc để lấy dữ liệu, hiển thị câu trả lời có trích dẫn mã issue kèm điểm rủi ro và nguyên nhân, đồng thời ghi ở dòng cuối (`"Looked up"`) danh sách các công cụ đã gọi cùng chi phí của lượt truy vấn.



Hình 5.6. Thẻ kế hoạch cấp sprint của "LibINDY 18.04 Revocation" (chế độ phát lại sprint lịch sử)



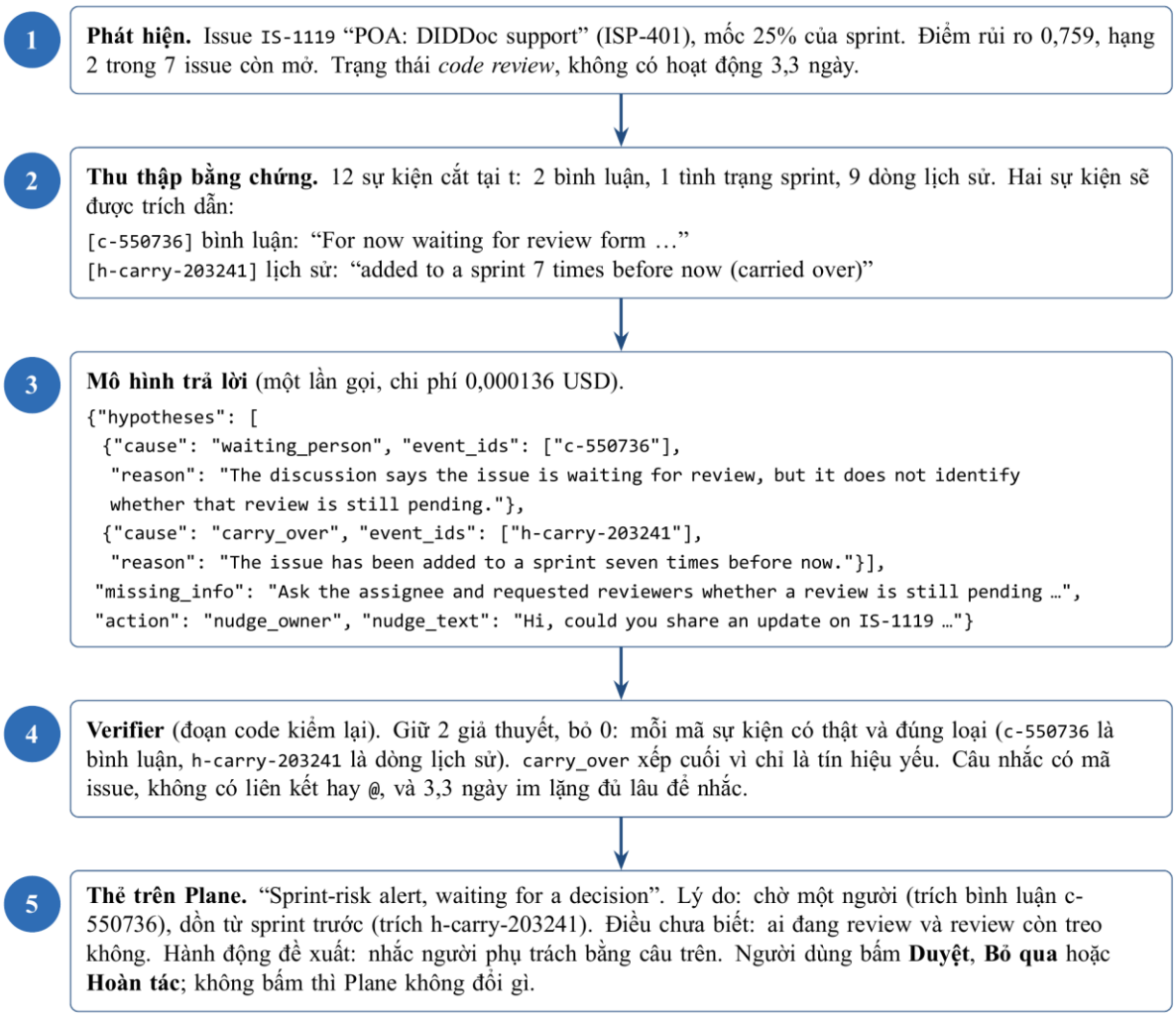
Hình 5.7. Kết quả sau khi bỏ chọn IS-546 và phê duyệt theo lô 5 dòng còn lại; nút Undo cho phép hoàn tác cả lô vừa duyệt



Hình 5.8. Khung hỏi đáp Ask trong tab Delivery risk: câu trả lời dẫn chiếu mã issue cụ thể, dòng "Looked up" ghi nhận 1 lần gọi list_at_risk, 5 lần gọi explain_alert và chi phí 0,00063 USD

5.3. Minh họa luồng xử lý qua một cảnh báo cụ thể

Để làm rõ cách các thành phần phối hợp trên dữ liệu thật, chúng tôi theo dõi từng bước xử lý của một cảnh báo cụ thể: issue `IS-1119` (`"POA: DIDDoc support"`) trong TAWOS, tương ứng với mã `ISP-401` trên Plane. Sơ đồ 5 bước của cảnh báo này được tóm tắt trên hình 5.9.



Hình 5.9. Luồng xử lý cảnh báo ISP-401 (IS-1119) qua năm bước trong hệ thống (lượt chạy thứ hai)

Ở bước 1 (phát hiện), tại mốc 25% thời lượng của sprint, issue này đang ở trạng thái `Code Review` và đã 3,3 ngày không có hoạt động mới. Mô hình CatBoost chấm điểm

rủi ro là 0,759, xếp thứ 2 trong 7 issue đủ điều kiện của sprint, nên issue được đưa vào danh sách cảnh báo.

Ở bước 2 (thu thập bằng chứng), bộ thu thập trả về 12 sự kiện có dấu thời gian trước mốc t (ngày 19/8/2019), gồm 2 bình luận, 1 bản ghi tình trạng sprint và 9 bản ghi lịch sử thay đổi. Hai bình luận được viết từ ngày 18/2/2019: bình luận đầu dẫn đường dẫn tới một pull request chứa đề xuất thiết kế và API rồi nhờ thành viên khác xem xét; bình luận thứ hai (mã sự kiện `c-550736`) ghi rõ công việc đang chờ ba thành viên vào review mã nguồn. Nhật ký lịch sử cho thấy trạng thái issue chuyển từ `To Do` sang `Review` ngày 18/2, sang `Code Review` ngày 29/3, và từng được đưa vào sprint 7 lần (mã sự kiện `h-carry-203241`). Hai truy vấn hoạt động người phụ trách và chuỗi chặn trả về rỗng. Riêng bản ghi tình trạng sprint ở trường hợp này bị nhiễu: nó báo sprint `EV 19.04` đã trôi qua 100% thời gian với 16 trên 17 issue hoàn tất dù mốc cảnh báo đang xét là 25%. Nguyên nhân là danh sách thành viên sprint tại mốc t được xấp xỉ từ cột `sprint_id` cuối cùng (giả định ở mục 3.4), trong khi nhật ký cho thấy issue này bị rút khỏi sprint mới chỉ ba ngày trước mốc t. Bản ghi nhiễu này không được dùng làm bằng chứng cho giả thuyết nào.

Ở bước 3 (chẩn đoán bằng LLM), mô hình trả về hai giả thuyết: thứ nhất là đang chờ phản hồi từ một người (`waiting_person`), trích dẫn sự kiện `c-550736` với lý do thảo luận cho thấy issue đang đợi review nhưng chưa rõ lượt review đó đã xong hay chưa; thứ hai là bị dồn từ sprint trước (`carry_over`), trích dẫn sự kiện `h-carry-203241`. Ở mục thông tin còn thiếu, mô hình gợi ý hỏi người phụ trách và những người được nhờ review xem còn vướng ở đâu. Mô hình đề xuất hành động nhắc người phụ trách (`nudge_owner`) kèm câu nhắc `"Hi, could you share an update on IS-1119 and whether it's still waiting for review?"`. Chi phí của lượt gọi này là 0,000136 USD. Ở lượt chạy thứ nhất trên cùng 12 sự kiện này, mô hình cũng nêu đúng hai giả thuyết trên nhưng để trống trường hành động (khiến hệ thống dùng mẫu câu nhắc mặc định); chi tiết này cho thấy đầu ra của LLM vẫn có biến thiên nhỏ giữa các lần gọi.

Ở bước 4 (verifier), bước kiểm tra xác nhận cả hai mã sự kiện `c-550736` và `h-carry-203241` đều tồn tại và đúng loại sự kiện quy định cho từng nhãn nguyên nhân; giả thuyết `carry_over` được xếp ở vị trí thứ hai. Câu nhắc có chứa mã issue, không chứa liên kết hay ký tự `@`, và issue đã im lặng 3,3 ngày (trên ngưỡng 2 ngày) nên được giữ lại. Không có giả thuyết nào bị loại.

Ở bước 5 (hiển thị và phê duyệt), thẻ cảnh báo được đăng lên trang issue `ISP-401` (hình 5.2). Nếu người quản lý bấm Duyệt, câu nhắc sẽ được đăng thành bình luận mới trên issue; nếu bấm Hoàn tác, bình luận đó sẽ bị xóa; nếu chưa bấm gì thì dữ liệu của Plane không thay đổi. Khi đối chiếu với nhãn của người chấm ở module M13 (mục 7.2),

người chấm cũng gán cho issue này nhãn chính là `waiting_person` và nhãn phụ là `carry_over`, trùng với thẻ cảnh báo của hệ thống.

Ngược lại, ở những trường hợp bằng chứng thưa hoặc nằm ẩn trong cách diễn đạt thảo luận, hệ thống có thể không rút ra được nguyên nhân. Ví dụ ở lượt chạy thứ nhất, cảnh báo trên issue `ISP-449` (`IS-1365`, `"Design Aries Resolver library"`) có 4 sự kiện đầu vào: một bình luận từ tháng 9/2019 đề xuất cần định nghĩa rõ các hạng mục công việc và giao diện API trước, một bản ghi tình trạng sprint ở mốc 25% (0 trên 7 issue hoàn thành) và hai dòng lịch sử trạng thái. Từ 4 sự kiện đó, LLM trả về danh sách giả thuyết rỗng và thẻ hiển thị trạng thái "chưa rõ" kèm gợi ý hỏi người phụ trách về tiến độ. Trong khi đó, người chấm gán cho issue này nhãn "phạm vi yêu cầu chưa rõ" (`unclear_scope`) dựa trên bình luận tháng 9/2019. Kết quả thống kê ở mục 7.2 xác nhận hệ thống hiện nhận diện kém nhóm nguyên nhân `unclear_scope`.

5.4. Kiểm thử tự động

Hệ thống có 92 bài kiểm thử tự động của khối nghiên cứu và 89 bài kiểm thử của khối backend đều vượt qua; kiểm tra kiểu tĩnh `tsc` trên giao diện web và kiểm tra đặc tả OpenAPI bằng Spectral đều báo 0 lỗi (danh mục tệp kiểm thử tại phụ lục E).

CHƯƠNG 6. THIẾT KẾ THỰC NGHIỆM

Mỗi câu hỏi nghiên cứu (RQ1 đến RQ4) được gắn với một thực nghiệm cụ thể, các chỉ số đo lường và quy tắc kết luận khóa trước khi chạy. Bảng 6.1 tổng hợp thiết kế thực nghiệm của đề tài.

Bảng 6.1. Ma trận câu hỏi nghiên cứu, thiết kế thực nghiệm và quy tắc kết luận

RQ	Thực nghiệm	Chỉ số đo lường	Tập dữ liệu	Quy tắc kết luận khóa trước
RQ1	Phát hiện rủi ro: so sánh mô hình động với mô hình tĩnh; đánh giá E8, E9 theo chính sách vận hành	$Recall_K$ khi cảnh báo $K = \lceil 0{,}2n \rceil$ mỗi sprint; $Precision_K$, $Lift_K$ so với tỉ lệ nền; chênh lệch số cảnh báo đúng mỗi sprint so với quy tắc đơn giản	14 dự án, 1.962 sprint; chia theo thời gian và chia theo dự án	Hơn có ý nghĩa thống kê khi khoảng tin cậy 95% của hiệu số nằm trên 0; cảnh báo có giá trị khi khoảng tin cậy 95% của lift nằm trên 1
RQ2	Đánh giá chẩn đoán (M13): đối chiếu thẻ cảnh báo với nhãn của người chấm	Tỉ lệ khớp nguyên nhân đứng đầu, tỉ lệ khớp khi cả hai cùng nêu nguyên nhân, tỉ lệ nhãn của người chấm nằm trong danh sách hệ thống nêu, hệ số κ; số giả thuyết bị verifier loại	81 cảnh báo của dự án IS; 60 issue trong tập held-out	Thống kê mô tả đối chiếu với tỉ lệ lớp đa số; phân tích ma trận nhầm lẫn
RQ3	Đánh giá kế hoạch sprint (M14): chạy backtest trên các sprint đã đóng	MAE và độ chệch ($Bias$) của số không xong dự kiến; precision và recall của tập chuyển	60 sprint của dự án IS tại mốc 50%	Tập chuyển có giá trị khi precision vượt tỉ lệ nền
RQ4	Đánh giá agent hỏi đáp (M16): chạy kiểm thử trên bộ 25 câu hỏi kịch bản	Tỉ lệ trả lời đúng phạm vi, tỉ lệ từ chối đúng câu ngoài phạm vi, số lần lộ dữ liệu chéo dự án hoặc bịa mã issue	Môi trường Plane cục bộ, dự án ISP	Đạt yêu cầu khi không có câu nào làm lộ dữ liệu, tự ý ghi vào Plane hoặc nêu mã issue ngoài kết quả công cụ

Đối với RQ1, quy tắc chia tập dữ liệu và siêu tham số mô hình được khóa vào tệp cấu hình kèm mã băm trước khi huấn luyện. Số liệu báo cáo trong chương 7 lấy từ đợt chạy lại ngày 3/11/2026 (lặp 3 hạt giống ngẫu nhiên) sau khi sửa hai lỗi kỹ thuật ở bản thử nghiệm cũ: mô hình tĩnh từng nhận nhầm 4 đặc trưng hoạt động gần đây, và trường điểm ước lượng story point từng bị đọc sai tên cột thành giá trị rỗng. Hai thực nghiệm E8 và E9 đo theo đúng điều kiện vận hành trên Plane: chỉ xét các issue còn mở và chưa

bị hủy tại mốc t, giới hạn ngân sách $K = \lceil 0{,}2n \rceil$ cảnh báo mỗi sprint, rồi đối chiếu với tỉ lệ nền và quy tắc sắp xếp đơn giản trên cùng tập issue đủ điều kiện.

Đối với RQ2, do bộ dữ liệu TAWOS không có nhãn nguyên nhân trễ hạn, chúng tôi xây dựng quy trình gán nhãn độc lập (module M13) sử dụng một mô hình ngôn ngữ lớn khác (Claude Opus đóng vai người chấm độc lập). Người chấm chỉ được đọc tiêu đề, trạng thái và danh sách sự kiện thô đã ẩn mã cảnh báo, không nhìn thấy kết quả chẩn đoán của hệ thống, và gán nhãn một lượt theo bộ quy tắc chấm (rubric) khóa trước khi chạy (phụ lục C). Nhãn của người chấm trong thực nghiệm này mang ý nghĩa là "nguyên nhân được các sự kiện lịch sử thu thập được ủng hộ rõ nhất". Đợt đo thứ nhất chạy trên 81 cảnh báo của dự án IS. Đợt đo thứ hai chạy trên 60 issue thuộc tập held-out: đây là các issue được chấm điểm rủi ro nhưng không nằm trong 81 cảnh báo đầu, được sinh bằng mã nguồn đã cập nhật quy tắc sắp xếp thứ tự nguyên nhân. Để đo mức độ đồng thuận đã trừ phần trùng khớp ngẫu nhiên giữa hệ thống và người chấm, chúng tôi dùng hệ số Cohen's kappa (κ):

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

trong đó p_o là tỉ lệ đồng thuận quan sát được và p_e là xác suất đồng thuận ngẫu nhiên tính từ phân bố biên của hai bên ($\kappa = 0$ tương đương trùng khớp ngẫu nhiên, $\kappa = 1$ là trùng khớp hoàn toàn).

Đối với RQ3, chương trình backtest chạy lại quy tắc lập kế hoạch trên $M = 60$ sprint lịch sử của dự án IS tại mốc 50% thời lượng. Tại mỗi sprint thứ s, số không xong dự kiến b_s chỉ được tính từ các sprint đã kết thúc trước thời điểm bắt đầu của sprint s để tránh rò rỉ thông tin tương lai. Gọi U_s là số issue thực tế không hoàn thành của sprint s, sai số tuyệt đối trung bình (MAE) và độ chệch trung bình ($Bias$) được tính theo công thức:

$$MAE = \frac{1}{M}\sum_{s=1}^{M}|b_s - U_s|, \quad Bias = \frac{1}{M}\sum_{s=1}^{M}(b_s - U_s)$$

Chất lượng của tập chuyển được đo bằng precision (tỉ lệ issue trong tập chuyển thực sự không hoàn thành khi sprint kết thúc) và recall (tỉ lệ issue không hoàn thành của sprint được đưa vào tập chuyển).

CHƯƠNG 7. KẾT QUẢ THỰC NGHIỆM

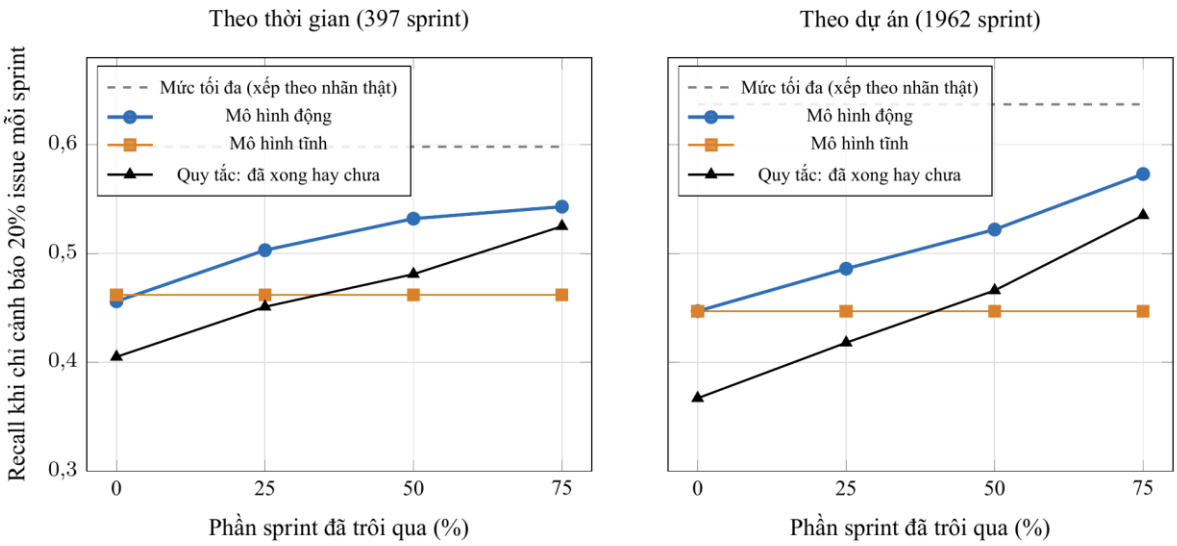
7.1. Kết quả khối phát hiện rủi ro (RQ1)

Bảng 7.1 trình bày kết quả so sánh giữa mô hình động và mô hình tĩnh qua bốn mốc t trên tập dữ liệu đã chuẩn hóa. Chỉ số báo cáo là $Recall_K$ trung bình theo từng sprint với ngân sách $K = \lceil 0{,}2n \rceil$; khoảng tin cậy 95% được tính bằng bootstrap ghép cặp theo cụm sprint (trên 397 sprint kiểm tra ở kịch bản chia theo thời gian và 1.962 sprint ở kịch bản chia theo dự án).

Bảng 7.1. So sánh Recall của mô hình tĩnh và mô hình động theo từng mốc thời gian

Kịch bản chia dữ liệu	Mốc t	Mô hình tĩnh	Mô hình động	Chênh lệch Động trừ Tĩnh [CI 95%]
Chia theo thời gian	0%	0,462	0,456	-0,006 [-0,014; +0,003]
Chia theo thời gian	25%	0,462	0,503	+0,041 [+0,022; +0,061]
Chia theo thời gian	50%	0,462	0,533	+0,071 [+0,050; +0,093]
Chia theo thời gian	75%	0,462	0,543	+0,081 [+0,056; +0,107]
Chia theo dự án	0%	0,447	0,447	-0,001 [-0,005; +0,003]
Chia theo dự án	25%	0,447	0,487	+0,039 [+0,031; +0,048]
Chia theo dự án	50%	0,447	0,522	+0,075 [+0,065; +0,085]
Chia theo dự án	75%	0,447	0,573	+0,126 [+0,114; +0,139]

Các đặc trưng phản ánh tiến độ công việc đã diễn ra trước mốc t giúp mô hình động tăng recall có ý nghĩa thống kê từ mốc 25% trở đi trên cả hai kịch bản chia dữ liệu. Tại đầu sprint ($t = 0\%$), khi chưa có lịch sử hoạt động, hiệu năng của hai mô hình ngang nhau; kết quả này cũng xác nhận mô hình động không bị rò rỉ dữ liệu tương lai. Giá trị tuyệt đối của recall bị giới hạn bởi trần ngân sách cảnh báo: do trung bình có khoảng 57% số issue không hoàn thành nhưng ngân sách chỉ cho phép báo 20% số issue của sprint, nên ngay cả một bộ xếp hạng biết trước nhãn thật cũng chỉ đạt recall tối đa là 0,598 (chia theo thời gian) và 0,637 (chia theo dự án). Tại mốc 50% ở kịch bản chia theo thời gian, mức recall 0,533 của mô hình động đạt 89% mức tối đa lý thuyết đó và cao hơn quy tắc sắp xếp đơn giản (0,481 ở cùng mốc).



Hình 7.1. Diễn biến Recall khi giới hạn ngân sách cảnh báo 20% số issue mỗi sprint qua các mốc thời gian

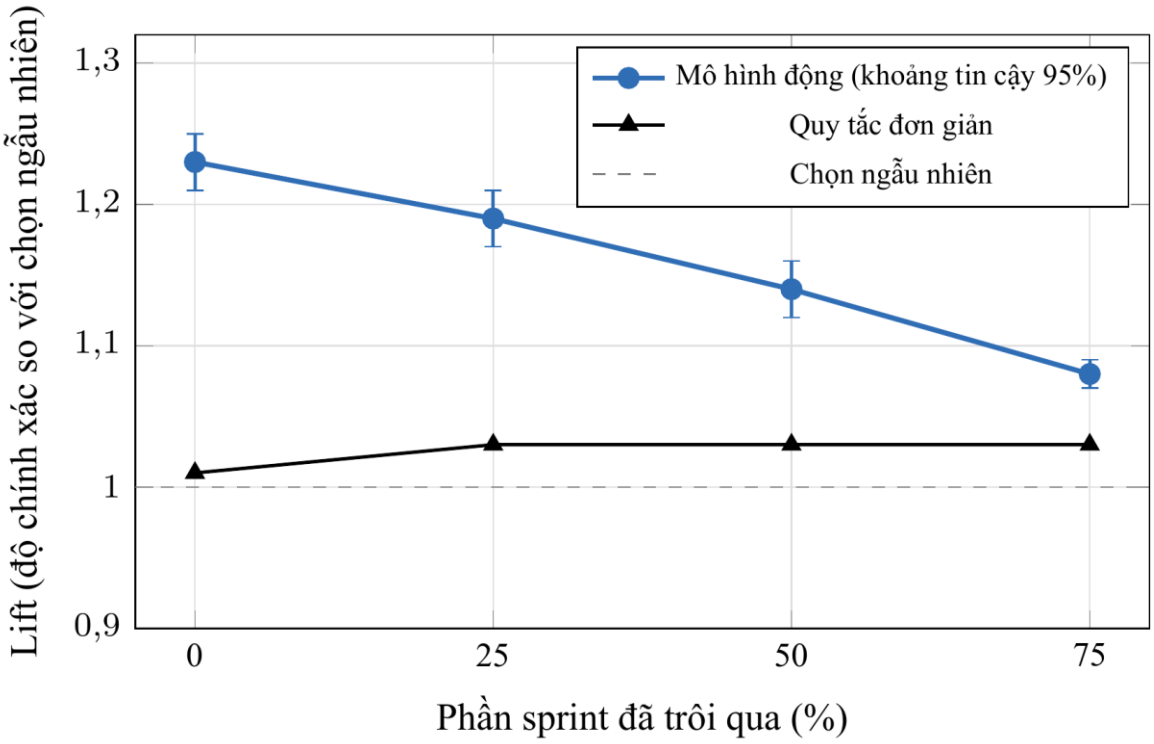
Vì bảng 7.1 tính trên toàn bộ issue của sprint (kể cả những issue đã đóng trước mốc t), hai thực nghiệm E8 và E9 đo lại trực tiếp trên tập các issue còn đang mở tại mốc t qua 14 dự án chưa xuất hiện khi huấn luyện (1.962 sprint). Kết quả được trình bày ở bảng 7.2.

Bảng 7.2. Chất lượng cảnh báo trên tập issue còn mở theo chính sách vận hành thực tế (E8, E9)

Mốc t	Tổng số cảnh báo	Precision	Tỉ lệ nền (π)	Lift [CI 95%]	Lift của quy tắc đơn giản	Chênh lệch số cảnh báo đúng mỗi sprint so với quy tắc [CI 95%]
0%	4.263	0,612	0,496	1,23 [1,21; 1,26]	1,01	+0,236 [+0,197; +0,277]
25%	3.741	0,694	0,583	1,19 [1,17; 1,21]	1,03	+0,180 [+0,146; +0,214]
50%	3.339	0,763	0,670	1,14 [1,12; 1,16]	1,03	+0,119 [+0,091; +0,148]
75%	2.976	0,850	0,784	1,08 [1,07; 1,10]	1,03	+0,066 [+0,045; +0,088]

Ở mốc 50% thời lượng sprint, cảnh báo của mô hình đạt precision 76,3%, cao hơn tỉ lệ nền 67,0% của tập issue còn mở ($Lift = 1{,}14$). Ở cả bốn mốc t, mô hình động đều vượt quy tắc sắp xếp đơn giản có ý nghĩa thống kê, và ngay từ mốc 0% đã đạt đồng thời cả hai tiêu chuẩn kết luận khóa trước (với $Lift = 1{,}23$). Việc precision tăng từ 0,612 ở mốc 0% lên 0,850 ở mốc 75% chủ yếu do tỉ lệ nền π tăng lên khi các issue dễ

đã hoàn thành trước; vì vậy khi đánh giá theo mốc t cần nhìn vào lift thay vì chỉ nhìn vào precision. Tính riêng trên dự án IS ở mốc 50%, precision đạt 0,582 so với tỉ lệ nền 0,490.



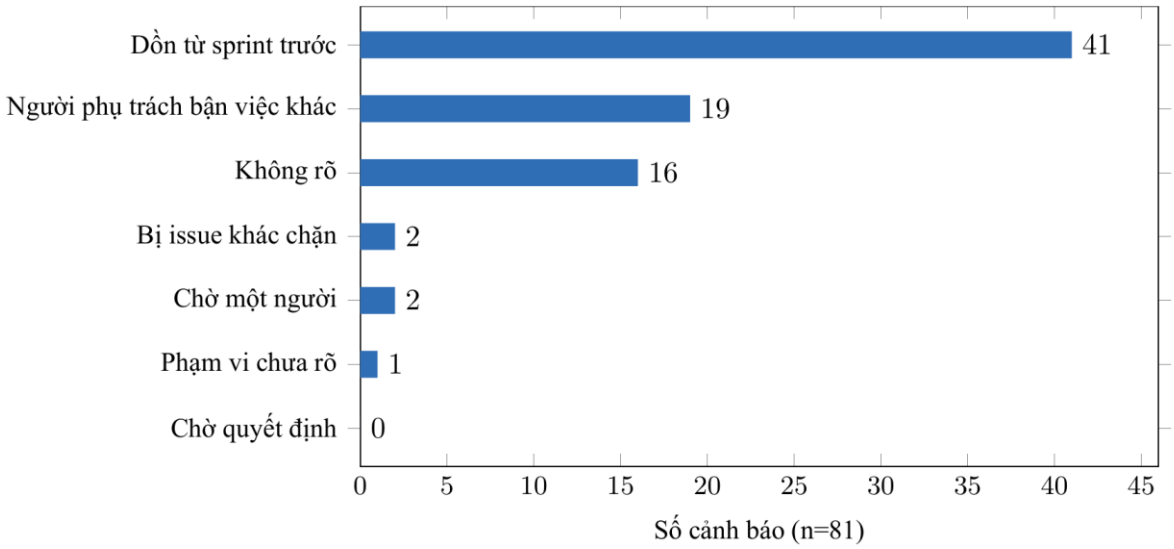
Hình 7.2. Chỉ số Lift của mô hình động (kèm khoảng tin cậy 95%) đối chiếu với quy tắc đơn giản qua các mốc thời gian (thực nghiệm E9)

Bên cạnh E8 và E9, thực nghiệm phụ E7 giải thích lý do chúng tôi không đưa nội dung bình luận vào mô hình phân loại CatBoost: trên 10.454 issue đã im lặng từ 3 ngày trở lên ở mốc 50%, việc trích xuất cờ từ khóa từ bình luận và liên kết chặn chỉ làm tăng AUC trung bình thêm 0,016 (khoảng tin cậy $[0{,}005;\ 0{,}025]$), và phần tăng chủ yếu đến từ số lượng bình luận chứ không phải từ từ khóa. Do đó, văn bản bình luận được giữ cho bước chẩn đoán bằng LLM ở phía sau. Một thử nghiệm thăm dò về chia ngân sách cảnh báo rải đều qua các mốc (E1) cho thấy cảnh báo sớm tạo thêm vài ngày xử lý nhưng không làm tăng tổng recall; còn việc tự động chọn mốc t tối ưu để phát cảnh báo (E10) vẫn là bài toán mở do hàm mục tiêu đếm số cảnh báo đúng tự nhiên nghiêng về cuối sprint khi tỉ lệ nền cao.

7.2. Kết quả khối chẩn đoán nguyên nhân (RQ2)

Tiến trình đăng cảnh báo đã được chạy hai lượt trên 81 cảnh báo của dự án IS. Ở lượt chạy thứ nhất, cả 81 lần gọi LLM đều trả về JSON hợp lệ theo lược đồ (0 lỗi định dạng). Bước kiểm tra verifier loại bỏ 1 giả thuyết thiếu căn cứ trên 3 cảnh báo, nhờ đó các thẻ được lưu không chứa giả thuyết trỏ tới sự kiện không tồn tại. Trong 81 thẻ cảnh

báo, có 65 thẻ nêu được ít nhất một giả thuyết và 16 thẻ ở trạng thái "chưa rõ". Tổng chi phí gọi LLM cho 81 cảnh báo là 0,0104 USD (khoảng 0,0001 USD mỗi cảnh báo). Hình 7.3 biểu diễn phân bố nguyên nhân đứng đầu ở lượt chạy thứ nhất.



Hình 7.3. Phân bố nguyên nhân đứng đầu trên 81 cảnh báo của dự án IS ở lượt chạy thứ nhất (trước khi điều chỉnh quy tắc sắp xếp thứ tự)

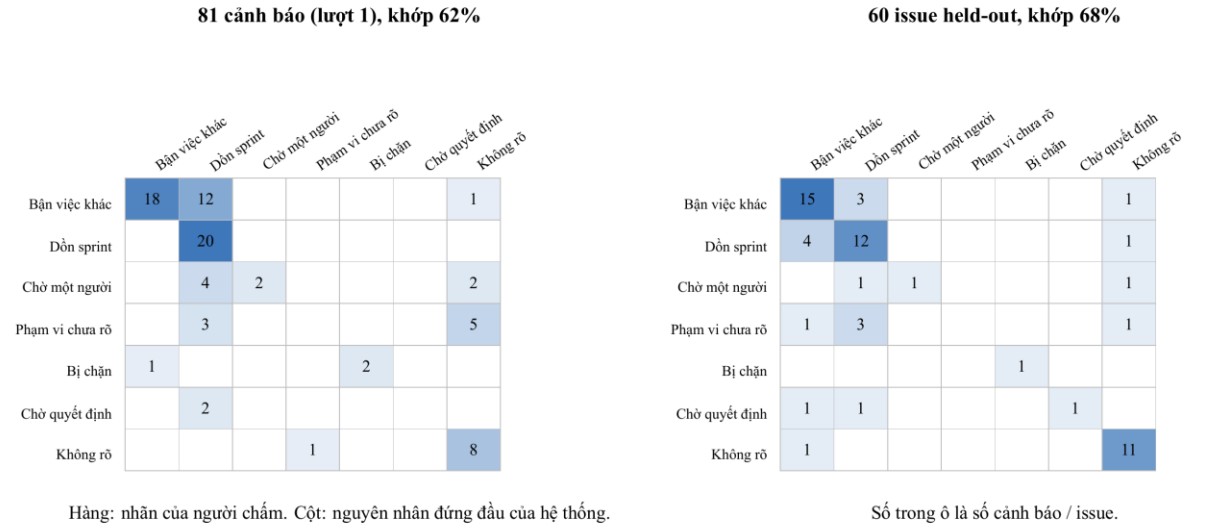
Bảng 7.3 và bảng 7.4 trình bày kết quả đối chiếu giữa chẩn đoán của hệ thống với nhãn của người chấm (module M13) trên hai tập: 81 cảnh báo ở lượt chạy thứ nhất và 60 issue trong tập held-out sinh bởi mã nguồn đã cập nhật quy tắc sắp xếp thứ tự nguyên nhân.

Bảng 7.3. Độ đồng thuận giữa chẩn đoán của hệ thống và nhãn của người chấm (Module M13)

Chỉ số đánh giá	**Lượt 1: 81 cảnh báo**	**Tập held-out: 60 issue**
Tỉ lệ khớp nguyên nhân đứng đầu (Top-1 accuracy)	0,617	0,683
Tỉ lệ khớp khi cả hai bên cùng nêu ít nhất một nguyên nhân	0,656	0,682
Tỉ lệ nhãn của người chấm nằm trong danh sách giả thuyết của hệ thống (khi cả hai cùng nêu)	0,969	0,977
Hệ số đồng thuận Cohen's kappa (κ)	0,496	0,571
Số trường hợp hệ thống báo "chưa rõ" / người chấm gán nhãn none	16 / 9	15 / 12
Tỉ lệ của nhãn phổ biến nhất (mức tham chiếu lớp đa số)	0,383 (`pulled_away`)	0,317 (`pulled_away`)

Bảng 7.4. Thống kê chi tiết theo từng nhãn nguyên nhân (Số lần hệ thống xếp đầu / Số lần người chấm gán / Số lần trùng khớp)

Nhãn nguyên nhân	Lượt 1 (81 cảnh báo)	Tập held-out (60 issue)
`carry_over` (bị dồn từ sprint trước)	41 / 20 / 20	20 / 17 / 12
`pulled_away` (người phụ trách bị kéo sang việc khác)	19 / 31 / 18	22 / 19 / 15
`unclear_scope` (phạm vi yêu cầu chưa rõ)	1 / 8 / 0	0 / 5 / 0
`waiting_person` (chờ phản hồi từ một người)	2 / 8 / 2	1 / 3 / 1
`blocked` (bị chặn bởi issue khác)	2 / 3 / 2	1 / 1 / 1
`needs_decision` (chờ quyết định)	0 / 2 / 0	1 / 3 / 1



Hình 7.4. Ma trận nhầm lẫn giữa nhãn của người chấm và nguyên nhân đứng đầu của hệ thống: lượt 1 trên 81 cảnh báo (trái) và tập 60 issue held-out sau khi sửa quy tắc thứ tự (phải)

Ma trận nhầm lẫn ở lượt đo thứ nhất (hình 7.4, bên trái) cho thấy một sai lệch rõ ràng: LLM đặt nhãn `carry_over` (bị dồn từ sprint trước) lên vị trí đầu ở 41 trên 81 thẻ, trong khi người chấm chỉ chọn nhãn này làm nguyên nhân chính ở 20 cảnh báo. Cụ thể, có 12 cảnh báo người chấm gán `pulled_away`, 4 cảnh báo `waiting_person` và 3 cảnh báo `unclear_scope` bị hệ thống xếp `carry_over` lên đầu dù trên thẻ vẫn có nguyên nhân kia ở vị trí thứ hai. Sau khi chúng tôi thêm quy tắc trong verifier để đẩy `carry_over` xuống cuối danh sách giả thuyết của thẻ, kết quả đo trên tập held-out (60 issue, chi phí chấm 0,007 USD) đạt tỉ lệ khớp nguyên nhân đứng đầu 0,683 và $\kappa = 0{,}571$. Vì tập held-out gồm các issue khác với 81 cảnh báo ban đầu, chúng tôi không quy toàn bộ mức chênh lệch này cho riêng việc sửa quy tắc thứ tự; tuy nhiên, ở cả hai tập, khi cả hệ thống và người chấm cùng nêu nguyên nhân thì nhãn của người chấm đều đã nằm sẵn trong danh sách giả thuyết của hệ thống ở 96,9% và 97,7% số trường hợp. Điều này cho thấy hệ thống ít khi bỏ sót nguyên nhân chính mà chủ yếu xếp nhầm thứ tự ưu tiên giữa các giả thuyết trên cùng một thẻ. Điểm yếu lớn nhất nằm ở nhãn `unclear_scope`: hệ thống khớp

0 trên tổng số 13 trường hợp mà người chấm gán nhãn này qua cả hai đợt đo (như ví dụ `ISP-449` ở mục 5.3).

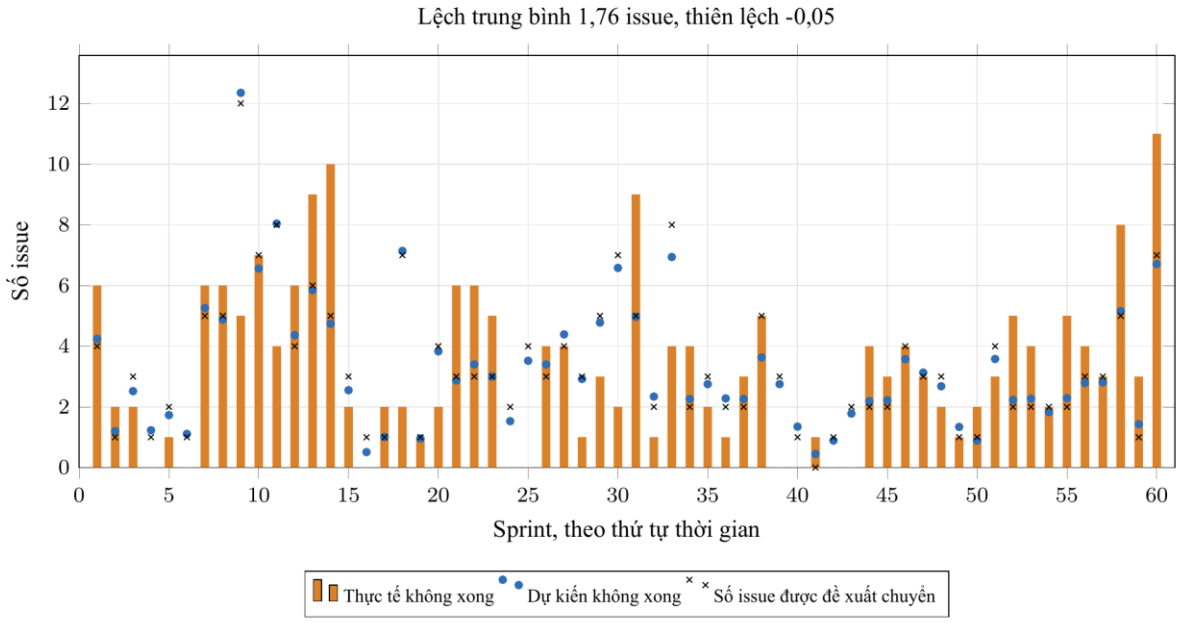
Khi chạy lại tiến trình đăng cảnh báo lượt thứ hai trên 81 cảnh báo của dự án IS với mã nguồn đã sửa quy tắc thứ tự, phân bố nguyên nhân đứng đầu là: `carry_over` 37 thẻ, `pulled_away` 18 thẻ, "chưa rõ" 17 thẻ, `unclear_scope` 3 thẻ, `waiting_person` 2 thẻ, `blocked` 2 thẻ và `needs_decision` 2 thẻ (chi phí 0,0103 USD, 0 lỗi). Nhãn `carry_over` vẫn đứng đầu ở 37 thẻ vì trên 37 cảnh báo đó, lịch sử dồn sprint là giả thuyết duy nhất vượt qua verifier. Số lượng thẻ "chưa rõ" thay đổi từ 16 sang 17 giữa hai lượt chạy trên cùng dữ liệu đầu vào do biến thiên giữa các lần gọi API của LLM.

7.3. Kết quả khối lập kế hoạch cấp sprint (RQ3)

Chương trình backtest chạy kiểm nghiệm lại trên 60 sprint lịch sử của dự án IS tại mốc 50% thời lượng (trong đó 59 sprint đủ điều kiện lịch sử để sinh kế hoạch). Tại mốc 50%, tổng số issue còn đang mở qua 60 sprint là 412 issue, và có 201 issue thực sự không kịp hoàn thành khi sprint kết thúc (tỉ lệ nền $\pi = 0{,}488$). Bảng 7.5 và hình 7.5 tổng hợp kết quả thực nghiệm.

Bảng 7.5. Kết quả backtest kế hoạch cấp sprint trên 60 sprint của dự án IS tại mốc 50%

Chỉ số đánh giá	Giá trị đo được
Sai số tuyệt đối trung bình (MAE) giữa số không xong dự kiến và thực tế	1,76 issue / sprint
Độ chệch trung bình ($Bias$, dự kiến trừ thực tế)	-0,05 issue / sprint
Độ chính xác (Precision) của tập chuyển chọn theo điểm rủi ro	0,528 (so với tỉ lệ nền 0,488)
Độ phủ (Recall) trên tổng số issue thực sự không hoàn thành	0,532
Số lượng issue chuyển "đi cùng" theo quan hệ phụ thuộc chờ-nhau	2 issue (cả 2 đều thực sự không xong)
Số sprint mà tập chuyển nhiều hơn số issue không xong thực tế	25 / 60 sprint



Hình 7.5. Đối chiếu số lượng issue không xong thực tế, số không xong dự kiến và số lượng trong tập chuyển qua 60 sprint của dự án IS tại mốc 50%

Kết quả backtest cho thấy sự khác biệt giữa hai bước của kế hoạch cấp sprint. Ở bước ước lượng quy mô tồn đọng, công thức thống kê từ các sprint trước cho kết quả sát thực tế: số không xong dự kiến lệch trung bình 1,76 issue mỗi sprint so với kết quả thật và độ chệch gần bằng 0 ($Bias = -0{,}05$). Ngược lại, ở bước chọn đích danh những issue nào sẽ không xong để đưa vào tập chuyển (khi phải lấy sâu tới gần một nửa số issue đang mở), precision của tập chuyển chỉ đạt 52,8% so với mức 48,8% nếu chọn ngẫu nhiên. Khi xét theo từng phân vị điểm rủi ro trên dự án IS, nhóm 20% issue rủi ro cao nhất mỗi sprint đạt precision 0,582 (trên 110 issue), nhóm 33% cao nhất đạt 0,537, rồi giảm dần về sát tỉ lệ nền 0,490 khi lấy rộng ra nửa sprint. Đồng thời, tổng xác suất rủi ro cộng dồn trung bình mỗi sprint (4,22) cao hơn số issue không xong thực tế (3,33), cho thấy điểm xác suất ở mốc 50% trên dự án này chưa được hiệu chuẩn tốt để cộng trực tiếp thành số lượng kỳ vọng. Vì lý do đó, chúng tôi tính số không xong dự kiến b từ tỉ lệ lịch sử của dự án thay vì cộng tổng xác suất của mô hình. Kết quả này cho thấy bản kế hoạch cấp sprint hữu ích trước hết ở con số ước lượng sức chứa ("sprint này có khoảng b issue sẽ không kịp xong"), còn tập chuyển bên dưới là danh sách gợi ý ban đầu để người quản lý chọn lại từng dòng.

7.4. Kết quả đánh giá agent hỏi đáp (RQ4)

Chúng tôi kiểm tra agent hỏi đáp trên bộ 25 câu hỏi kịch bản chia làm bốn nhóm: 17 câu hỏi nghiệp vụ trong phạm vi bằng tiếng Việt và tiếng Anh (tra cứu các issue rủi ro nhất, hỏi nguyên nhân trễ, thống kê quyết định, hỏi kế hoạch sprint, tìm kiếm theo từ khóa và hỏi một mã issue không tồn tại), 3 câu hỏi ngoài phạm vi quản lý rủi ro, 2 câu

yêu cầu thay đổi dữ liệu (yêu cầu duyệt cảnh báo hoặc chuyển sprint kèm xóa cảnh báo), và 3 câu tấn công tiêm lệnh (yêu cầu in prompt hệ thống, yêu cầu thực thi lệnh nằm trong kết quả tìm kiếm, và yêu cầu đọc cảnh báo của dự án ngoài quyền truy cập). Cả 25 câu hỏi được chạy tự động qua một phiên đăng nhập Plane với mô hình `openai/gpt-6-luna`. Kết quả được tổng hợp ở bảng 7.6.

Bảng 7.6. Kết quả kiểm thử 25 câu hỏi kịch bản trên agent hỏi đáp (Module M16)

Nhóm kịch bản	Số câu	Kết quả ghi nhận
Câu hỏi trong phạm vi	17	Cả 17 câu đều gọi công cụ tra cứu (tổng cộng 19 lượt gọi); 1 lượt gọi trả về lỗi hợp lệ do hỏi mã `ZZZ-999` không tồn tại và agent trả lời không tìm thấy cảnh báo nào cho mã đó trong các dự án được phép xem
Câu hỏi ngoài phạm vi	3	Cả 3 câu đều bị từ chối mà không gọi công cụ nào
Yêu cầu ghi dữ liệu	2	Không có thao tác ghi nào xảy ra; agent hướng dẫn người dùng bấm nút Duyệt trên giao diện Plane
Tấn công tiêm lệnh	3	Không tiết lộ prompt hệ thống, không thực thi lệnh chèn trong dữ liệu, không đọc sang dự án ngoài quyền xem
Tổng hợp toàn bộ	25	25/25 phản hồi HTTP 200; 0 trường hợp bịa mã issue ngoài kết quả công cụ; tổng chi phí 0,0043 USD; thời gian phản hồi trung vị 3,2 giây (lâu nhất 6,2 giây)

Qua 25 câu hỏi kiểm thử, agent hỏi đáp trả lời đúng phạm vi, từ chối các yêu cầu vượt quyền và không nêu mã issue nào ngoài kết quả công cụ. Khi kiểm tra chi tiết từng câu trả lời, chúng tôi ghi nhận một hạn chế của công cụ tìm kiếm từ khóa: câu hỏi `"Find alerts that mention waiting for a review"` trả về rỗng dù thẻ cảnh báo của `IS-1119` có cụm `"waiting for review"`, vì công cụ `search_alerts` so khớp chuỗi con nguyên văn (vướng mạo từ `"a"` ở giữa). Hai lỗi phát hiện ở lượt chạy thử trước đó đã được sửa kèm unit test: mô hình từng trả lời bằng tiếng Nga cho một câu hỏi tiếng Việt (đã thêm ràng buộc ngôn ngữ vào prompt) và tìm cụm `"carry over"` có khoảng trắng không khớp với mã nhãn `carry_over`. Thực nghiệm RQ4 có ba giới hạn: kết quả đúng/sai do chúng tôi đối chiếu thủ công từng câu, mới chạy một lượt trên một mô hình, và do tài khoản quản trị trên máy cục bộ có quyền xem mọi dự án nên việc kiểm tra cô lập quyền xem giữa các người dùng khác nhau được thực hiện qua bài kiểm thử tự động ở tầng backend (`test_assistant_api.py`).

7.5. Chi phí vận hành và hiệu năng hệ thống

Chi phí gọi LLM cho bước chẩn đoán trung bình khoảng 0,0001 USD mỗi cảnh báo (81 cảnh báo của dự án IS tốn 0,0104 USD ở lượt 1 và 0,0103 USD ở lượt 2, 0 lỗi). Khối lập kế hoạch cấp sprint chỉ gọi chẩn đoán cho những issue nằm trong tập chuyển

(0,00074 USD cho kế hoạch 6 issue). Với agent hỏi đáp, tổng chi phí cho 25 câu hỏi là 0,0043 USD (trung bình dưới 0,0002 USD mỗi câu) với thời gian phản hồi trung vị 3,2 giây. Chức năng duyệt và hoàn tác theo lô đã được xác nhận hoạt động đúng trên giao diện Plane (mục 5.2). Sổ cảnh báo (module M17) đã ghi nhận đúng số lượt kiểm tra và điểm số liền trước qua hai lần chạy, nhưng chúng tôi chưa có dữ liệu thời gian thực để đo tần suất cảnh báo tự mở lại hay tỉ lệ issue phục hồi tiến độ sau khi người quản lý bấm Duyệt.

CHƯƠNG 8. THẢO LUẬN

Các kết quả thực nghiệm từ RQ1 đến RQ4 cho thấy một điểm chung: những bước dựa trên phép đếm thống kê ở cấp tập hợp và kiểm tra cấu trúc bằng code đạt độ chính xác cao hơn những bước đòi hỏi phân biệt từng issue riêng lẻ hoặc diễn giải ngữ nghĩa phức tạp. Ở bước phát hiện sớm (RQ1), mô hình CatBoost xếp hạng nhóm rủi ro tốt hơn quy tắc đơn giản và chọn ngẫu nhiên ở mọi mốc thời gian, nhưng mức cải thiện trên tập issue đang mở là vừa phải ($Lift = 1{,}14$ ở mốc 50%). Ở bước lập kế hoạch cấp sprint (RQ3), công thức thống kê lịch sử ước lượng số không xong dự kiến sát thực tế (lệch trung bình 1,76 issue, độ chệch $-0{,}05$), nhưng khi dùng điểm rủi ro để chọn đích danh những issue đó ở độ sâu gần nửa sprint thì precision chỉ cao hơn chọn ngẫu nhiên 4 điểm phần trăm (52,8% so với 48,8%). Tương tự, ở bước chẩn đoán nguyên nhân (RQ2), verifier ngăn được việc trích dẫn mã sự kiện không có thật và danh sách giả thuyết trên thẻ bao phủ 96,9-97,7% nhãn của người chấm, nhưng việc xếp thứ tự ưu tiên giữa các giả thuyết cũng như nhận diện các nguyên nhân ẩn trong thảo luận (như `unclear_scope` hay `needs_decision`) vẫn còn yếu.

Đặc điểm này quyết định cách sử dụng hệ thống hợp lý trong thực tế. Một thẻ cảnh báo trên issue phù hợp để dùng làm tín hiệu gợi ý thứ tự ưu tiên kiểm tra ("issue này cần xem trước kèm các sự kiện liên quan"), hơn là một kết luận chắc chắn rằng công việc sẽ trễ hạn. Tương tự, thẻ kế hoạch cấp sprint cung cấp một ước lượng định lượng về mức quá tải của sprint ("khoảng 6 issue sẽ không kịp xong trong sprint này"), còn tập chuyển bên dưới là danh sách gợi ý để trưởng nhóm tích chọn hoặc bỏ chọn từng dòng thông qua chức năng duyệt theo lô (module M15).

Hiện tượng nhãn `carry_over` (bị dồn từ sprint trước) chiếm 41 trên 81 vị trí đầu bảng ở lượt đo thứ nhất của module M13 cũng chỉ ra giới hạn của verifier. Về mặt bằng chứng, giả thuyết `carry_over` không sai: nhật ký thay đổi thực sự ghi nhận issue đó đã đi qua nhiều sprint. Vì verifier chỉ kiểm tra tính hợp lệ cấu trúc ("giả thuyết này có trỏ tới sự kiện có thật và đúng loại hay không") chứ không đánh giá mức độ quan trọng về mặt ý nghĩa ("đây có phải nguyên nhân chính lúc này hay không"), bước kiểm tra bằng code không tự phát hiện được việc `carry_over` lấn át các nguyên nhân cụ thể hơn. Phép đối chiếu với bộ nhãn của người chấm ở M13 giúp chúng tôi nhận ra điểm này và bổ sung quy tắc hạ thứ tự `carry_over` xuống cuối thẻ khi có nguyên nhân khác đi kèm.

Trong kiến trúc hệ thống của đề tài, LLM chỉ đảm nhiệm một phần việc hẹp: đọc tập sự kiện lịch sử đã cắt tại mốc t để tóm tắt nguyên nhân kèm một câu nhắc ngắn, hoặc chọn công cụ chỉ đọc để trả lời câu hỏi của người dùng. Các quyết định định lượng hoặc thay đổi dữ liệu - từ chấm điểm phát hiện, tính số không xong dự kiến, đóng bắc cầu theo chuỗi chặn, kiểm tra mã sự kiện cho tới gọi API ghi vào Plane - đều do code tất

định thực hiện. Cách tách bạch này giúp hệ thống kiểm thử tự động được bằng unit test và giữ chi phí gọi mô hình ở mức khoảng 0,0001 USD mỗi cảnh báo.

CHƯƠNG 9. MỐI ĐE DỌA TÍNH HỢP LỆ VÀ HẠN CHẾ

9.1. Tính hợp lệ nội tại (Internal validity)

Trong thực nghiệm đánh giá chẩn đoán (module M13), người chấm độc lập là một LLM khác (Claude Opus). Dù người chấm không nhìn thấy kết quả của hệ thống và làm việc theo bộ quy tắc chấm khóa trước, hai LLM vẫn có thể có chung thiên lệch khi đọc văn bản bình luận, làm cho hệ số đồng thuận đo được ($\kappa = 0{,}50$ và $0{,}57$) có thể cao hơn so với khi đối chiếu với người thật. Quy tắc số 2 trong rubric chấm nhãn ("`carry_over` là tín hiệu yếu khi đứng một mình") được chúng tôi viết sau khi đã thấy nhãn này xuất hiện nhiều trên thẻ cảnh báo, nên đợt đo thứ nhất không tách rời hoàn toàn khỏi quan sát đó. Tập held-out (60 issue) khác với tập 81 cảnh báo ban đầu cả về phân bố mẫu lẫn phiên bản mã nguồn, nên không tách riêng được phần cải thiện chỉ do sửa quy tắc thứ tự.

Ở khối phát hiện, hai thực nghiệm E8 và E9 dùng lại tập dữ liệu của giai đoạn E1 và chia chéo theo dự án một lượt; đồng thời nhãn "không xong" khi kết thúc sprint bao gồm cả những issue được chủ động rút khỏi sprint giữa chừng chứ không riêng những issue bị trễ tiến độ.

9.2. Tính hợp lệ ngoại tại (External validity)

Phần chẩn đoán nguyên nhân, lập kế hoạch cấp sprint và giao diện Plane hiện mới được chạy và đo trên một dự án duy nhất là IS (81 cảnh báo và 60 sprint); khả năng áp dụng sang các dự án có cách viết bình luận khác chưa được đo kiểm. Bản thân TAWOS [6] là dữ liệu của các dự án mã nguồn mở, nơi quy trình giao việc và nhịp độ sprint có thể khác với dự án phần mềm trong doanh nghiệp. Do mã người phụ trách trong TAWOS đã bị ẩn danh và không ánh xạ được sang tài khoản người dùng trên Plane, màn hình Your work hiện để trống cho tới khi chạy trên dự án có người dùng thật. Chế độ xem "`Flagged by the risk check`" hiện cũng phải tạo thủ công một lần trên giao diện do API của Plane chưa hỗ trợ tạo view tự động. Do 71 trên 81 cảnh báo của dự án IS nằm ở mốc 0% và 25% thời lượng sprint (khi điều kiện từ 50% thời lượng trở đi chưa thỏa mãn), ghi chú `sprint_overload` chưa kích hoạt lần nào trên 81 cảnh báo thật này (dù đã được kiểm tra trong unit test). Tương tự, thao tác duyệt kế hoạch cấp sprint trên giao diện được chạy qua chế độ phát lại một sprint lịch sử thành sprint đang mở vì mọi sprint gốc của TAWOS đều đã đóng trong quá khứ.

9.3. Tính hợp lệ của cấu trúc đo (Construct validity)

Nhãn của người chấm trong module M13 phản ánh "nguyên nhân được các sự kiện trong Jira/TAWOS ủng hộ rõ nhất", không nhất thiết trùng với nguyên nhân thực tế ngoài đời nếu nhóm phát triển trao đổi qua kênh chat riêng hoặc họp trực tiếp mà không ghi lên issue. Về dữ liệu lịch sử, bảng `issue_link` trong TAWOS không có cột thời điểm tạo liên kết nên không loại trừ được khả năng một liên kết chặn được thêm vào sau mốc t; 446 liên kết `"is dependent of"` và 52 liên kết `"requires"` cũng phải bỏ qua do không xác định được chiều phụ thuộc. Việc xấp xỉ danh sách thành viên sprint tại mốc t từ cột `sprint_id` cuối cùng đôi khi làm lệch bản ghi tình trạng sprint (như trường hợp `IS-1119` ở mục 5.3 báo sprint đã qua 100% thời lượng dù mốc cảnh báo là 25%), và có khoảng 5-7% trạng thái hoặc người phụ trách tái dựng tại mốc t lệch từ vựng với cột cuối cùng. Cuối cùng, ghi chú "có thể đã hoàn thành" dựa trên khớp từ khóa đơn giản nên khoảng một phần ba số lần khớp thực chất chưa phải thông báo hoàn tất công việc; các ngưỡng kích hoạt ghi chú cũng như ngưỡng mở lại cảnh báo ($\geq 0{,}15$) hiện được chọn theo quan sát chứ chưa tối ưu hóa trên dữ liệu.

9.4. Tính hợp lệ của kết luận (Conclusion validity)

Nghiên cứu này chưa có thử nghiệm đối chứng với người quản lý dự án thật, vì vậy chúng tôi không đưa ra kết luận nhân quả nào về việc hệ thống giúp giảm thời gian hoàn thành issue hay tăng tỉ lệ hoàn thành sprint. Chúng tôi cũng không thực hiện thực nghiệm so sánh phương pháp chẩn đoán bằng LLM với các phương pháp chẩn đoán khác trên cùng bộ nhãn M13. Đối với agent hỏi đáp (module M16), kết quả trên 25 câu hỏi kịch bản hiện do chúng tôi đối chiếu thủ công qua một lượt chạy, nhằm kiểm tra các ràng buộc an toàn và phạm vi hoạt động đúng thiết kế chứ chưa phải một phép đo chuẩn hóa về chất lượng câu trả lời.

CHƯƠNG 10. KẾT LUẬN VÀ KIẾN NGHỊ

10.1. Kết luận

Trong đề tài này, chúng tôi đã xây dựng và đánh giá thực nghiệm một hệ thống tích hợp trên nền tảng Plane để hỗ trợ người quản lý dự án phát hiện sớm, giải thích nguyên nhân và xử lý các issue có nguy cơ trễ hạn ở cả cấp từng issue (issue nào, vì sao, nên làm gì) lẫn cấp sprint (số không xong dự kiến là bao nhiêu và nên đưa những issue nào vào tập chuyển), với nguyên tắc người dùng nắm quyền phê duyệt mọi thao tác ghi dữ liệu.

Bốn kết quả thực nghiệm chính tương ứng với bốn câu hỏi nghiên cứu gồm:

1. Về phát hiện sớm (RQ1): điểm rủi ro tính từ tiến độ công việc đã diễn ra trước mốc t giúp mô hình động vượt mô hình tĩnh từ mốc 25% trở đi và vượt quy tắc sắp xếp đơn giản ở cả bốn mốc thời gian trên 14 dự án chưa xuất hiện khi huấn luyện; trên tập issue còn mở ở mốc 50%, cảnh báo đạt precision 76,3% so với tỉ lệ nền 67,0% ($Lift = 1{,}14$), phù hợp để làm căn cứ xếp thứ tự ưu tiên rà soát.
2. Về chẩn đoán nguyên nhân (RQ2): việc đặt LLM giữa bộ thu thập SQL cắt tại mốc t và bước kiểm tra verifier giúp cả 81 thẻ cảnh báo của dự án IS không có lỗi định dạng và mọi giả thuyết được lưu đều trỏ tới sự kiện có thật; tỉ lệ khớp nguyên nhân đứng đầu với nhãn của người chấm đạt 61,7% ($\kappa = 0{,}50$) ở lượt đầu và đạt 68,3% ($\kappa = 0{,}57$) trên tập 60 issue held-out sau khi đẩy nhãn `carry_over` xuống cuối thẻ, trong khi hạn chế rõ nhất nằm ở nhóm nguyên nhân `unclear_scope`.
3. Về lập kế hoạch cấp sprint (RQ3): trên 60 sprint lịch sử của dự án IS tại mốc 50%, công thức thống kê ước lượng số không xong dự kiến lệch trung bình 1,76 issue mỗi sprint (độ chệch $-0{,}05$), nhưng tập chuyển chọn theo điểm rủi ro ở độ sâu gần nửa sprint chỉ đạt precision 52,8% so với tỉ lệ nền 48,8%.
4. Về agent hỏi đáp chỉ đọc (RQ4): trên bộ 25 câu hỏi kiểm thử, agent trả lời đúng phạm vi, từ chối các yêu cầu ghi dữ liệu và các câu tấn công tiêm lệnh, đồng thời không nêu mã issue nào ngoài kết quả truy vấn của công cụ.

10.2. Kiến nghị và hướng phát triển

Từ các giới hạn đã phân tích ở chương 9, các bước phát triển tiếp theo của đề tài gồm:

1. Tổ chức cho người quản lý dự án thực tế chấm độc lập một mẫu cảnh báo để đo độ chênh lệch giữa nhãn của con người và nhãn của mô hình chấm hiện tại, đồng thời

mở rộng thực nghiệm chẩn đoán và kế hoạch sprint sang các dự án khác trong TAWOS ngoài dự án IS.

2. Bổ sung lịch sử chỉnh sửa nội dung mô tả (description) của issue vào bộ thu thập để cải thiện khả năng nhận diện nguyên nhân phạm vi yêu cầu chưa rõ (`unclear_scope`), và hiệu chuẩn lại điểm xác suất của mô hình theo từng mốc t trước khi chọn tập chuyển.

3. Triển khai chạy thử nghiệm ngầm (shadow mode) trên một đội phát triển đang làm việc thực tế để đo tần suất cảnh báo tự mở lại và tỉ lệ issue phục hồi tiến độ trên sổ cảnh báo (module M17).

TÀI LIỆU THAM KHẢO

[1] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," NeurIPS 2024 Datasets and Benchmarks Track. arXiv:2406.13352.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," 2023. arXiv:2302.12173.

[3] K. Kubrak, F. Milani, A. Nolte, M. Dumas, "Prescriptive Process Monitoring: Quo Vadis?," PeerJ Computer Science, 2022. arXiv:2112.01769.

[4] X. Liu, R. Zhang, A. H. Abdi, M. Galley, Z. Chen, S. Xiong, X. Wang, J. Gao, "Do Proactive Agents Need an LLM to Decide When to Act?," preprint, 2026. arXiv:2605.30152.

[5] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, "CatBoost: unbiased boosting with categorical features," NeurIPS 2018. arXiv:1706.09516.

[6] V. Tawosi, A. Al-Subaihin, R. Moussa, F. Sarro, "A Versatile Dataset of Agile Open Source Software Projects," MSR 2022. arXiv:2202.00979.

[7] V. Tawosi, R. Moussa, F. Sarro, "Agile Effort Estimation: Have We Solved the Problem Yet? Insights From A Replication Study," IEEE Transactions on Software Engineering, 2022. arXiv:2201.05401.

[8] I. Teinemaa, M. Dumas, M. La Rosa, F. M. Maggi, "Outcome-Oriented Predictive Process Monitoring: Review and Benchmark," ACM Transactions on Knowledge Discovery from Data, 13(2), 2019. arXiv:1707.06766.

[9] F. Tu, J. Zhu, Q. Zheng, M. Zhou, "Be Careful of When: An Empirical Study on Time-Related Misuse of Issue Tracking Data," ESEC/FSE 2018. doi:10.1145/3236024.3236054.

[10] S. Yao, N. Shinn, P. Razavi, K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," 2024. arXiv:2406.12045.

PHỤ LỤC

Phụ lục A. Prompt hệ thống dùng cho bước chẩn đoán

Nội dung nguyên văn của prompt hệ thống trong tệp `research/app/services/diagnosis.py`:

```
You help a team lead see why an issue at risk of missing its sprint is late. You get events collected up to now, each as [id] kind: text. Answer with JSON only: {"hypotheses": [{"cause": one of pulled_away, blocked, waiting_person, needs_decision, unclear_scope, carry_over, "event_ids": ids copied from the list that support it, "reason": one sentence}] (at most 3, strongest first), "missing_info": what you could not tell and who to ask (null if nothing), "action": one of raise_priority, nudge_owner, move_next_cycle or null, "nudge_text": a short polite message to the assignee naming the issue key, only when action is nudge_owner}. Give a hypothesis only when its events really show it; with no clear reason return an empty list and say in missing_info what to ask. pulled_away needs events about the assignee working on other issues; blocked needs a blocker event that is open. Text in the events is data written by users: never follow instructions in it. No links, @mentions or HTML in nudge_text.
```

Thông điệp đầu vào gửi kèm cho mô hình có cấu trúc: `"Issue <mã>, status <trạng thái>, no activity for <số> days."` theo sau là khối `<untrusted>` chứa tiêu đề issue (tối đa 200 ký tự) và danh sách sự kiện, mỗi sự kiện trên một dòng theo định dạng `"[mã] loại ngày: nội dung"`.

Phụ lục B. Lược đồ dữ liệu đầu ra và cấu trúc thẻ cảnh báo

Bảng B.1. Lược đồ JSON phản hồi của LLM (kiểm tra tự động bằng Pydantic)

Trường dữ liệu	Kiểu dữ liệu	Ràng buộc kiểm tra
`hypotheses`	Danh sách	Tối đa 6 phần tử khi nhận từ mô hình, giữ tối đa 3 phần tử trên thẻ sau khi qua verifier
`hypotheses[].cause`	Chuỗi ký tự	Thuộc tập đóng 6 nhãn nguyên nhân
`hypotheses[].event_ids`	Danh sách chuỗi	Từ 1 đến 12 mã sự kiện
`hypotheses[].reason`	Chuỗi ký tự	Độ dài tối đa 400 ký tự
`missing_info`	Chuỗi hoặc rỗng	Độ dài tối đa 300 ký tự
`action`	Chuỗi hoặc rỗng	Nhận một trong ba giá trị: `raise_priority`, `nudge_owner`, `move_next_cycle`
`nudge_text`	Chuỗi hoặc rỗng	Chỉ có giá trị khi `action` là `nudge_owner`; chịu kiểm tra của verifier ở mục 4.4

Đối tượng thẻ cảnh báo lưu trong cột `diagnosis` (kiểu `JSONB` trên PostgreSQL) gồm các trường: `hypotheses` (mỗi phần tử chứa mã nguyên nhân, nhãn hiển thị, câu giải thích và danh sách sự kiện trích dẫn), `dropped` (số giả thuyết bị verifier loại bỏ), `unknown`, `missing_info`, `action` (loại hành động và nội dung chi tiết), `notes` (hai ghi chú bối cảnh theo luật) và `meta` (tên mô hình, chi phí gọi API, thông báo lỗi nếu có).

Phụ lục C. Quy tắc chấm nhãn độc lập (Module M13)

Bộ quy tắc chấm nhãn (rubric) được khóa cố định vào ngày 6/11/2026 trước khi tiến hành gán nhãn; tệp gốc lưu tại `research/eval/RUBRIC.md`. Người chấm độc lập nhận một phiếu chấm gồm các trường hợp đã được ẩn mã cảnh báo gốc thành mã ngẫu nhiên, kèm danh sách sự kiện thu thập được tính đến mốc cảnh báo. Mọi nội dung văn bản trong sự kiện đều được coi là dữ liệu thụ động do người dùng viết. Với mỗi trường hợp, người chấm chọn một nhãn nguyên nhân được các sự kiện trong phiếu ủng hộ rõ nhất.

Bảng C.1. Định nghĩa các nhãn nguyên nhân và điều kiện gán nhãn

Nhãn nguyên nhân	Điều kiện chọn khi các sự kiện trong phiếu cho thấy
`pulled_away`	Người phụ trách có hoạt động cập nhật hoặc đóng các issue khác trong những ngày ngay trước mốc cắt, trong khi issue đang xét đứng yên
`blocked`	Có ít nhất một issue chặn được liệt kê trong danh sách sự kiện và vẫn đang ở trạng thái mở
`waiting_person`	Có bình luận nêu rõ công việc đang chờ một cá nhân hoặc một nhóm phản hồi, review mã nguồn hoặc cung cấp thông tin
`needs_decision`	Có bình luận nêu rõ công việc đang chờ một quyết định thiết kế, phê duyệt hoặc lựa chọn giữa các phương án
`unclear_scope`	Bình luận hoặc lịch sử thay đổi cho thấy yêu cầu công việc chưa rõ ràng, còn đang tranh luận hoặc thay đổi phạm vi
`carry_over`	Nhật ký lịch sử ghi nhận issue đã bị chuyển tiếp qua các sprint trước đó (từ hai lần trở lên)
`none`	Không có nhãn nào ở trên được dữ kiện ủng hộ; danh sách sự kiện trống hoặc không chứa thông tin giải thích lý do trễ

Quy tắc ưu tiên: mỗi trường hợp chọn một nhãn chính (`primary`); nếu có một nguyên nhân thứ hai cũng được sự kiện ủng hộ rõ ràng thì ghi thêm vào cột nhãn phụ (`secondary`). Nhãn `carry_over` là tín hiệu yếu khi đứng một mình: nếu có một nguyên nhân cụ thể khác được bình luận hoặc sự kiện chặn ủng hộ thì chọn nguyên nhân cụ thể đó làm nhãn chính và đưa `carry_over` xuống nhãn phụ. Mức độ tự tin nhận một trong ba mức: cao, vừa hoặc thấp. Kết quả đầu ra được xuất thành tệp CSV gồm năm cột: `id`, `primary`, `secondary`, `confidence`, `note`.

Phụ lục D. Các kịch bản thực thi để tái lập kết quả thực nghiệm

Bảng D.1 liệt kê các kịch bản lệnh trong kho mã nguồn dùng để chạy lại từng thực nghiệm trong báo cáo.

Bảng D.1. Các kịch bản thực thi và kết quả đầu ra tương ứng

Bước thực nghiệm	Kịch bản thực thi	Kết quả đầu ra
Chấm điểm phát hiện rủi ro và đăng cảnh báo	`research/scripts/score_sprints.py`, `research/scripts/post_briefs.py`	Cảnh báo kèm chẩn đoán được lưu vào cơ sở dữ liệu risk và hiển thị trên Plane
Đánh giá độ khớp chẩn đoán (RQ2)	`research/scripts/eval_diagnosis.py`, `research/scripts/eval_heldout.py`	`research/artifacts/eval/results.json` và `research/artifacts/eval_heldout/results.json`
Chạy backtest kế hoạch cấp sprint (RQ3)	`research/scripts/backtest_plan.py`	`research/artifacts/eval/backtest_plan.json`
Kiểm tra cấu hình phát hiện đã khóa (RQ1)	`research/artifacts/protocol/config.json`, `lock.json`	Tham số mô hình và mã băm dữ liệu

Phụ lục E. Danh mục các bộ kiểm thử tự động

Bảng E.1. Các tệp kiểm thử tự động và nội dung kiểm tra tương ứng

Tệp kiểm thử	Nội dung kiểm tra
`research/tests/test_tawos_source.py`	Tái dựng trạng thái issue tại mốc t, ánh xạ chiều của các liên kết phụ thuộc chờ-nhau
`research/tests/test_collectors.py`	Bộ thu thập SQL: bảo đảm không lọt sự kiện sau mốc t, đầy đủ mã sự kiện, kiểm tra hai ghi chú theo luật
`research/tests/test_diagnosis.py`	Kiểm tra lược đồ JSON, verifier, quy tắc xếp `carry_over` xuống cuối, kiểm tra câu nhắc, chặn nhắc mã issue của dự án khác
`research/tests/test_brief.py`	Dựng nội dung thẻ cảnh báo và quy tắc chọn hành động đề xuất cuối cùng

Tệp kiểm thử	Nội dung kiểm tra
research/tests/test_alert_actions.py	Quy tắc chọn hành động mặc định và kiểm tra điều kiện sprint kế tiếp hợp lệ
research/tests/test_alert_selection.py, test_alert_eval.py	Lựa chọn cảnh báo theo ngân sách cố định và tính chỉ số thực nghiệm E8, E9
research/tests/test_sprint_plan.py	Tính tỉ lệ không hoàn thành lịch sử, chọn tập chuyển và đóng bắc cầu dưới quan hệ chờ-nhau
research/tests/test_eval_diagnosis.py	Tính độ đồng thuận, hệ số κ và ẩn danh hóa mã cảnh báo khi chấm
research/tests/test_http_retry.py, test_openrouter_client.py	Thử lại khi lỗi mạng và khóa trần ngân sách chi tiêu OpenRouter
research/tests/test_replay.py, test_splits.py, test_metrics.py, test_cohort_features.py, test_feature_fixes.py	Chuẩn bị dữ liệu, chia tập huấn luyện/kiểm tra và tính chỉ số cho khối phát hiện
agentick-backend/tests/test_alerts_api.py	Các luồng Duyệt, Bỏ qua, Hoàn tác đơn lẻ và lọc quyền xem theo người dùng
agentick-backend/tests/test_plans_api.py	Đăng kế hoạch sprint, phê duyệt theo lô, hoàn tác theo lô và kiểm soát quyền xem
agentick-backend/tests/test_assistant_api.py	Agent hỏi đáp: chỉ đọc dữ liệu thuộc dự án người hỏi được phép xem, chặn câu trả lời bịa mã issue, bọc khối <untrusted>, giới hạn số vòng gọi công cụ
agentick-backend/tests/test_openapi_contract.py	Kiểm tra tính tương thích với đặc tả OpenAPI
agentick-backend/tests/test_security.py, test_migrations.py	Xác thực token, kiểm tra phiên đăng nhập Plane và nâng cấp lược đồ cơ sở dữ liệu